



Universidad
de Huelva

Universidad de Huelva
Facultad de Ingeniería Informática

Memoria Vellum

Programación de juegos 3D en Unity

Autor:

Matteo Sonaglioni

Año Académico 2025-2026

Curso de Programación de Juegos

Índice

1	Introducción	6
1.1	Descripción general	6
1.2	Objetivo del jugador	7
1.3	Estructura del proyecto	8
2	Estructuras de Escenas (Flujo de Juego)	10
2.1	Menú Principal y Pausa	10
2.2	Prólogo: el libro y el narrador	11
2.3	Acto 1 — El Camino (puzzle de memoria)	12
2.4	Acto 2 — La Arena y la Estatua	13
2.5	Acto 3 — El Espejo de Agua	14
2.6	Interfaces de estado y HUD	15
3	Algoritmos, Inteligencia Artificial y Comportamiento	17
3.1	Controlador del Jugador	17
3.2	Pipeline de combate orientado a datos	18
3.3	Motor de Inferencia Difusa Mamdani	18
3.4	Generación Procedimental de Caminos (Acto 1)	21
3.5	Steering y Cerco Angular (Acto 2)	22
3.6	Máquina de Estados del Jefe + Defensa Reactiva (Acto 3)	23
3.7	El compañero Jammo	24
3.8	Cinemáticas, cámaras y “flip” del mundo	24
4	Arquitectura del Código	26
4.1	Carpeta: Player	26
4.1.1	PlayerCombat.cs	26
4.1.2	PlayerMeleeAttack.cs	27
4.1.3	MeleeFinisherState.cs	27
4.1.4	PlayerHealth.cs	28
4.1.5	PlayerSkinSwitcher.cs	28
4.1.6	AnimationEventForwarder.cs	29
4.2	Carpeta: Combat	29
4.2.1	Health.cs	29
4.2.2	DamageInfo.cs	31
4.2.3	IDamageable.cs	31
4.2.4	IDamageFilter.cs	31

4.2.5	IDamageReaction.cs	31
4.2.6	KnockbackReceiver.cs	31
4.2.7	FrontalShieldBlock.cs	33
4.3	Carpeta: AI/Fuzzy	33
4.3.1	FuzzyController.cs	33
4.3.2	FuzzyVariable.cs	34
4.3.3	FuzzyRule.cs	34
4.3.4	MembershipFunction.cs	35
4.4	Carpeta: Enemies	35
4.4.1	EnemyAI.cs	35
4.4.2	EnemyFuzzyBrain.cs	36
4.4.3	EnemyTargetCoordinator.cs	37
4.5	Carpeta: Level_1 (Acto 1 — Puzzle del Camino)	38
4.5.1	PathPuzzleManager.cs	38
4.5.2	PathTile.cs	40
4.5.3	DoorPortal.cs	41
4.5.4	DoorBuildController.cs	42
4.5.5	JammoGuideController.cs	43
4.5.6	PathGeneration/SelfAvoidingPathGenerator.cs	44
4.5.7	PathGeneration/FuzzyPathEvaluator.cs	46
4.5.8	PathGeneration/GridPath.cs	47
4.6	Carpeta: Level_2 (Acto 2 — Arena y Estatua)	47
4.6.1	WaveManager.cs	47
4.6.2	StatueAssemblyDirector.cs	49
4.6.3	StatueRig.cs	50
4.6.4	PieceSpawner.cs	51
4.6.5	PickupRotator.cs	51
4.6.6	JammoCarrier.cs	52
4.6.7	JammoHealth.cs	53
4.6.8	JammoPartSet.cs	54
4.6.9	Act02Director.cs	55
4.6.10	VortexEnterTransition.cs	56
4.7	Carpeta: Level_3 (Acto 3 — El Espejo de Agua)	56
4.7.1	MirrorDuelDirector.cs	56
4.7.2	BossDuelAI.cs	58
4.7.3	BossFuzzyBrain.cs	60
4.7.4	BossShield.cs	62

4.7.5	DuelHealthSpawner.cs	63
4.7.6	MirrorFlipDirector.cs	63
4.7.7	EnemySkinMirror.cs	64
4.8	Carpeta: JammoCompanion	65
4.8.1	JammoCompanion.cs	65
4.8.2	JammoActivationTrigger.cs	66
4.9	Carpeta: Items	66
4.9.1	InteractableObject.cs	66
4.9.2	InteractionUIManager.cs	67
4.9.3	Items/Objects/BookManager.cs	67
4.9.4	Items/Objects/CinematicFallManager.cs	67
4.9.5	Items/Objects/HealthPickup.cs	67
4.9.6	Items/Objects/DuelHealthPickup.cs	67
4.10	Carpeta: Dialogue	68
4.10.1	DialogueLine.cs	68
4.10.2	DialogueAsset.cs	68
4.10.3	DialogueManager.cs	68
4.11	Carpeta: UI	68
4.11.1	PlayerHUD.cs	68
4.11.2	HudReveal.cs	68
4.11.3	StatueProgressBar.cs	68
4.11.4	MirrorCameraSync.cs	69
4.11.5	FloorProximityManager.cs	69
4.11.6	CRTController.cs	69
4.11.7	SettingsUI.cs	69
4.11.8	GlobalButtonStyling.cs	70
4.11.9	ButtonStyleOverride.cs	71
4.12	Carpeta: Score	71
4.12.1	RunStats.cs	71
4.12.2	Leaderboard.cs	71
4.12.3	ScoreManager.cs	72
4.12.4	ScoreLevelsBinder.cs	73
4.12.5	LeaderboardUI.cs	73
4.12.6	LeaderboardEntryView.cs	74
4.13	Carpeta: Audio	74
4.13.1	SoundManager.cs	74
4.13.2	AudioCue.cs	74

4.13.3	LevelMusic.cs	74
4.13.4	PlayerCombatAudio.cs	74
4.14	Carpeta: Menu	74
4.14.1	MainMenuManager.cs	74
4.15	Carpeta: Utils	76
4.15.1	SimplePool.cs	76
4.15.2	AnimatorParameterCache.cs	76
4.15.3	TileGridGenerator.cs	76
4.15.4	UIRaycastDebugger.cs	76
4.15.5	Utils/Editor/ForestSower.cs	76
4.15.6	Utils/Editor/PathTileSetupTool.cs	77
4.15.7	Utils/Editor/SceneViewCapture.cs	77
4.15.8	PathPuzzleManagerEditor.cs	77
5	Agradecimientos y Referencias de Activos	78

1 Introducción

Este documento detalla el diseño y desarrollo de un videojuego de acción-aventura **en 3D**, creado con el motor **Unity 6** sobre la **Universal Render Pipeline (URP)** y el lenguaje **C#**. El proyecto se distingue de un juego de niveles repetitivos por estar concebido como un **escaparate de sistemas y algoritmos diferenciados**: en lugar de reutilizar el mismo bucle de juego en cada nivel, cada uno de los tres *Actos* implementa una mecánica central y una técnica de ingeniería distinta. A nivel de inteligencia artificial el proyecto incorpora un **motor de lógica difusa de Mamdani** construido desde cero, **generación procedimental de caminos** (backtracking + heurística de Warnsdorff), **steering y cerco de multitudes** y una **máquina de estados finitos** con defensa reactiva para el jefe final.

1.1 Descripción general

Vellum es un juego de **acción-aventura narrativa en tercera persona**, de un solo jugador y de progresión lineal. El protagonista avanza acompañado de **Jammo**, un pequeño robot que es mecánicamente central en dos de los tres niveles (ensambla una estatua en el Acto 2 y transporta piezas de reliquia en el Acto 3). El juego abre con un **prólogo** (un libro antiguo y un narrador que habla en pasado) y se despliega a través de tres Actos autocontenidos:

- **Acto 1 — El Camino:** un puzzle de memoria sobre una rejilla de baldosas, con un camino correcto generado proceduralmente en cada partida.
- **Acto 2 — La Arena y la Estatua:** una arena de combate por oleadas en la que hay que proteger a Jammo mientras él reconstruye una estatua con las piezas que sueltan los enemigos.
- **Acto 3 — El Espejo de Agua:** un duelo final contra un doble (*doppelganger*) del jugador, en dos fases idénticas (Sol y Luna) separadas por una transición de mundo.

La base técnica es **URP + Cinemachine 3 + el nuevo Input System** (con StarterAssets para la locomoción en tercera persona). El estilo de combate es **cuerpo a cuerpo** con un sistema de **combo → finisher** sobre una capa enmascarada del Animator y un estado de **defensa/escudo frontal**.



Figura 1: Portada/escena de presentación del juego o del prólogo

1.2 Objetivo del jugador

El objetivo inmediato del jugador cambia con cada Acto, lo que sostiene la curva de aprendizaje:

1. **Acto 1:** memorizar y recorrer el camino generado para cruzar la sala. El suelo global se apaga al iniciar el puzzle: solo las baldosas correctas sostienen al jugador, de modo que pisar una equivocada provoca una caída y un reaparecimiento tras un breve retardo. Una **pista de memoria** de cargas limitadas ilumina los siguientes tramos correctos. Al completar el camino, una puerta se construye sola y se abre.
2. **Acto 2:** sobrevivir y **proteger a Jammo** el tiempo suficiente para que complete la estatua. La arena no termina por número de oleadas, sino cuando la **estatua está completa** (las oleadas se repiten en bucle hasta entonces).
3. **Acto 3:** **ganar el duelo de dos fases.** En cada fase, golpear al enemigo desbloquea piezas que Jammo lleva al altar del jugador; cuando todas están entregadas, el enemigo se vuelve vulnerable (hasta el 50% en la fase Sol, hasta 0 = victoria en la fase Luna).

A nivel global, el desempeño del jugador (tiempo, precisión, daño recibido, bloqueos perfectos, rachas, piezas perdidas...) se traduce en una **puntuación final** con una **nota sintética (S/A/B/C/D)** y se registra en una **tabla de clasificación** persistente (Score/), permitiendo competir por un puesto. La partida se pierde si mueren el jugador o Jammo.

1.3 Estructura del proyecto

Todo el código escrito a mano vive en `Assets/Scripts/`; todo lo que cuelga de `Assets/Downloaded/` es material de terceros y no se modifica a mano. El repositorio sigue una separación por responsabilidades: se distinguen el código fuente (organizado en submódulos funcionales: jugador, combate, IA, niveles, UI, audio, puntuación...), los recursos audiovisuales (modelos, animaciones, audio, shaders) y la configuración del motor.

A continuación se presenta el árbol de directorios principal del código, con una breve descripción del propósito de cada carpeta:

```
Vellum/
├── Assets/
│   └── Scripts/
│       ├── Player/
│       │   └── NÚCLEO DE LA APLICACIÓN
│       │       └── # Personaje del jugador
│       │           ├── PlayerCombat.cs           # Combo + finisher por capa enmascarada, defensa
│       │           ├── PlayerMeleeAttack.cs      # Ventanas de golpe / detección de impacto
│       │           ├── MeleeFinisherState.cs     # StateMachineBehaviour del finisher
│       │           ├── PlayerHealth.cs          # Vida del jugador + engancho de Game Over
│       │           ├── PlayerSkinSwitcher.cs     # Hot-swap del modelo + Animator.Rebind()
│       │           └── AnimationEventForwarder.cs # Reenvío de eventos de animación de StarterAssets
│       ├── Combat/
│       │   └── # Sistema de combate compartido y orientado a datos
│       │       ├── Health.cs                     # Componente de vida para todos los actores
│       │       ├── IDamageable.cs / DamageInfo.cs
│       │       ├── IDamageReaction.cs / IDamageFilter.cs
│       │       ├── KnockbackReceiver.cs         # Retroceso del enemigo golpeado
│       │       └── FrontalShieldBlock.cs        # Bloqueo frontal del jugador
│       ├── AI/Fuzzy/
│       │   └── # Motor de inferencia difusa de Mamdani (propio)
│       │       ├── FuzzyController.cs           # Motor + builder fluido
│       │       └── FuzzyVariable.cs / FuzzyRule.cs / MembershipFunction.cs
│       ├── Enemies/
│       │   └── # IA de la arena (Acto 2)
│       │       ├── EnemyAI.cs                   # Steering, cerco en abanico, agresión difusa
│       │       ├── EnemyFuzzyBrain.cs           # Distancia/vida/aglomeración -> agresión
│       │       └── EnemyTargetCoordinator.cs     # Slots angulares, objetivo Jugador/Jammo
│       ├── Level_1/
│       │   └── # Acto 1 - Puzzle del camino
│       │       ├── PathPuzzleManager.cs         # Flujo del puzzle, pistas, fallo/reaparición
│       │       └── PathGeneration/
│       │           ├── SelfAvoidingPathGenerator.cs # Backtracking + Warnsdorff
│       │           ├── FuzzyPathEvaluator.cs      # Puntuación estética difusa
│       │           └── GridPath.cs
```

— PathTile.cs / DoorBuildController.cs / DoorPortal.cs	
— JammoGuideController.cs	
— Level_2/	# Acto 2 - Arena y Estatua
— WaveManager.cs	# Oleadas, pooling, distribución de spawn, drops
— StatueAssemblyDirector.cs	# Jammo ensambla la estatua
— StatueRig.cs / PieceSpawner.cs / PickupRotator.cs	
— JammoCarrier.cs / JammoHealth.cs / JammoPartSet.cs	
— Act02Director.cs	# Cinemáticas de prólogo/epílogo -> Act_03
— VortexEnterTransition.cs	
— Level_3/	# Acto 3 - El Espejo de Agua
— MirrorDuelDirector.cs	# Orquestación del duelo Sol/Luna en dos compases
— BossDuelAI.cs	# FSM del jefe + defensa reactiva
— BossFuzzyBrain.cs	# Decisión difusa de fase Luna
— BossShield.cs / DuelHealthSpawner.cs	
— MirrorFlipDirector.cs	# Flip de cielo/mundo (skybox + capas)
— EnemySkinMirror.cs	
— JammoCompanion/	# Robot compañero (activación/seguir)
— JammoCompanion.cs / JammoActivationTrigger.cs	
— Items/	# InteractableObject + InteractionUIManager
— Objects/	# BookManager, CinematicFallManager, pickups
— Dialogue/	# DialogueAsset / DialogueLine / DialogueManager
— UI/	# PlayerHUD, StatueProgressBar, HudReveal, # CRTController, SettingsUI, estilizado de botones
— Score/	# RunStats, Leaderboard, ScoreManager, binders, UI
— Audio/	# SoundManager, AudioCue, LevelMusic, PlayerCombatAudio
— Menu/	# MainMenuManager
— Utils/	# SimplePool, AnimatorParameterCache, # TileGridGenerator (+ herramientas de Editor)

Nota. ProjectSettings/, Packages/, Library/, Temp/, Logs/, obj/ y docs/ son auto-generadas o de configuración del motor y quedan fuera del alcance de esta memoria.

2 Estructuras de Escenas (Flujo de Juego)

Para garantizar un rendimiento óptimo y una arquitectura limpia, el juego se divide en **escenas** siguiendo el principio de **responsabilidad única**. El flujo de ejecución es lineal: **Menú** → **Prólogo** → **Acto 1** → **Acto 2** → **Acto 3**. Las transiciones entre Actos se realizan con cargas de escena (`SceneManager.LoadScene`) enmarcadas en cinemáticas (efectos de “vórtice”, *flip* de cielo, etc.), de modo que el cambio sea fluido para el jugador.

A diferencia del proyecto 2D —donde un único `GameManager` instanciaba las arenas dentro de una sola escena—, aquí cada Acto es una escena propia (`Act_01`, `Act_02`, `Act_03`) con su propio **director** (un guión por corrutinas que orquesta cámaras, diálogos y reglas). Esto es coherente con la naturaleza narrativa del juego.

2.1 Menú Principal y Pausa

El menú principal, la **pausa en partida** y la **pantalla de fin de juego** están gobernados por un único script, `MainMenuManager`. Se adopta un enfoque de **panel único** (los submenús son `GameObjects` que se activan/desactivan con animaciones de *fade/slide*) en lugar de escenas distintas, lo que da transiciones instantáneas.

- **Navegación y transiciones.** Al pasar de menú a juego, `MainMenuManager` desactiva la cámara de menú, ajusta el *blend* de **Cinemachine**, desvanece el panel y desbloquea al jugador. La primera partida muestra el botón **Play** y lanza el diálogo de introducción tras asentarse la cámara; a partir de ahí, **Esc** alterna pausa (con `Time.timeScale = 0`) mostrando **Resume** y **Restart**. La pausa por **Esc** queda **bloqueada** mientras un diálogo, el libro o una cinemática están activos (referencias a `DialogueManager`, `BookManager`, `CinematicFallManager`, `JammoGuideController`).
- **Subpaneles.** Ajustes (`SettingsUI`) y tabla de clasificación (`LeaderboardUI`) se abren con *fade* y son mutuamente excluyentes.
- **Fin de partida.** `ShowGameOver()` (muerte) y `ShowVictory()` (victoria final) comparten la misma pantalla de fin (contenedor con campo de nombre + “Save Score”), cambiando solo el título (“GAME OVER” / “YOU WIN”). Antes de mostrarla se calcula la puntuación de la partida (`ScoreManager.ComputeScores`) y se muestran el **score** y la **nota**.

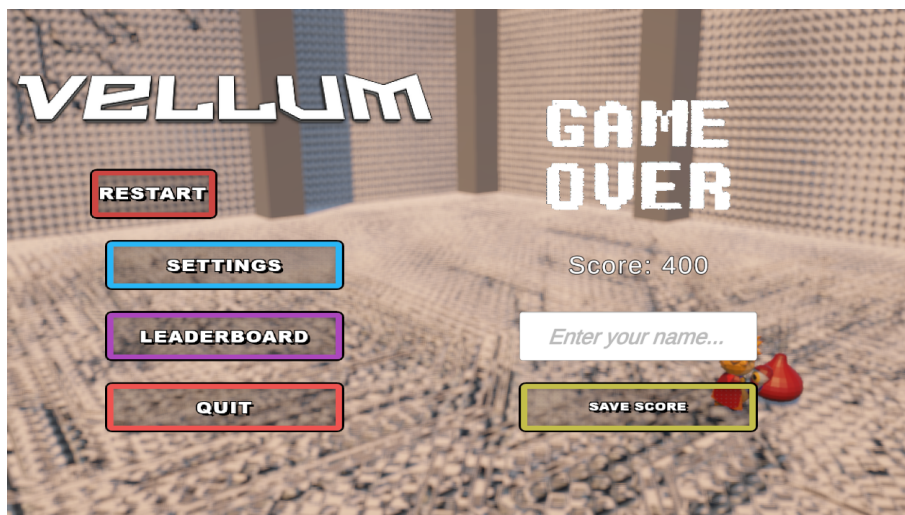


Figura 2: Pantalla de fin de partida (Game Over / Victory) con score y nota

2.2 Prólogo: el libro y el narrador

El prólogo introduce la historia con un **libro** del mundo y un **narrador** en pasado. El libro lo gestiona **BookManager**: la primera interacción “recoge” el libro (lo elimina del mundo) y a partir de entonces el jugador puede **abrir/cerrar el panel del libro** en cualquier momento con B/F; abrirlo bloquea al jugador y el primer cierre puede disparar un diálogo posterior. Los diálogos son **orientados a datos**: un **DialogueAsset** (lista de **DialogueLine**) reproducido por **DialogueManager**, que escribe el texto con efecto de máquina de escribir (Espacio salta/avanza) y **bloquea/desbloquea al jugador** durante la conversación.



Figura 3: Prólogo — libro abierto

2.3 Acto 1 — El Camino (puzzle de memoria)

La escena `Act_01` contiene una **rejilla de baldosas** (`PathTile`) generada por un utilitario de Editor (`TileGridGenerator`) sobre un *Plane*, más un anillo decorativo. El puzzle lo coordina `PathPuzzleManager`:

- Al **interactuar con Jammo** (`JammoCompanion`) se reproduce un diálogo de activación, Jammo se “levanta” y **comienza la secuencia del puzzle**: se activa la validación, se **apaga el suelo global** (solo desactiva sus colisionadores, no las baldosas) y Jammo recorre como **guía** el camino correcto (`JammoGuideController`), iluminando su rastro morado mientras el jugador queda bloqueado.
- El jugador debe pisar las baldosas en el **orden correcto**. Hay un **periodo de gracia** tras cada respawn/diálogo (las baldosas equivocadas solo se iluminan, no hacen caer), un **checkpoint** a mitad de camino (índice 36 de 73) y reaparición tras fallo.
- Una **pista de memoria** (tecla Q, cargas limitadas) ilumina los siguientes N tramos correctos durante unos segundos y parpadea antes de apagarse.
- Al completar el camino, una **puerta se construye por altura** mediante un shader (`DoorBuildController`, propiedad `_BuildHeight`) y se abre; cruzarla dispara una transición de **vórtice** (`DoorPortal`: giro de cámara + *lens distortion* + desaturación) y carga el Acto 2.



Figura 4: Acto 1 — Jammo guía iluminando el rastro; puerta construyéndose

2.4 Acto 2 — La Arena y la Estatua

La escena Act_02 es una **arena de combate por oleadas**, coordinada por Act02Director:

1. **Prólogo:** la cámara sube hacia la estatua (transición lenta de Cinemachine) y se reproduce la narración de introducción; luego vuelve al jugador y se revela el HUD.
2. **Combate:** WaveManager lanza oleadas de enemigos desde los muros (con pooling y distribución de spawn). Las oleadas **se repiten en bucle**; la arena no termina por número de oleadas.
3. **Ensamblaje:** en paralelo, StatueAssemblyDirector hace que **Jammo recoja piezas** (una por enemigo abatido), las lleve al pedestal y rellene un hueco de la estatua (StatueRig, PieceSpawner, JammoCarrier). La arena **termina cuando la estatua está completa**.
4. **Epílogo:** al completarse, la cámara vuelve a la panorámica, hay una narración final y se carga el **Acto 3**.

Si **Jammo muere** mientras transporta una pieza (puede recibir daño), se pierde el progreso de esa pieza; si su vida llega a 0 se produce **Game Over** (JammoHealth). La entrada a la escena usa un **vórtice inverso** (VortexEnterTransition) que revierte el efecto del portal del Acto 1.



Figura 5: Acto 2 — Jammo llevando una pieza al pedestal

2.5 Acto 3 — El Espejo de Agua

La escena Act_03 es el **duelo final**, orquestado por `MirrorDuelDirector`. El enemigo es un **doble del jugador** del sexo opuesto (`EnemySkinMirror`) y la arena se refleja sobre un plano de agua (`MirrorCameraSync`). El duelo tiene **dos fases idénticas (Sol → Luna)**, cada una en dos compases:

1. **Recolección:** el enemigo no es invulnerable, pero recibe 1 HP por golpe y no baja del suelo de la fase. Cada `hitsPerPiece` golpes del jugador (contados por `BossShield`) **desbloquean** una pieza colocada en el altar enemigo, que **Jammo lleva** al altar del jugador.
2. **Ventana de daño:** entregadas todas las piezas, el enemigo se vuelve vulnerable. En la fase **Sol** baja hasta el 50%; en la fase **Luna** hasta 0 (= **Victoria**).

Entre fases hay una **transición**: los tres actores caminan a sus puntos de aparición, se giran hacia el Sol/Luna y se ejecuta el **flip de cielo** (`MirrorFlipDirector`: rota los cuerpos celestes, hace *crossfade* de luces y **cambia el skybox y las capas UpWorld/DownWorld** en vez de un desvanecido duro). En la **fase Luna** el jefe está potenciado y puede **interrumpir su ataque para interceptar a Jammo** mientras transporta (decisión difusa, `BossFuzzyBrain`); si lo golpea, la pieza vuelve al altar enemigo y Jammo recibe daño. La victoria dispara un **epílogo** que **disuelve el mundo** alrededor del jefe (efecto de shader global) y muestra la pantalla de victoria.



Figura 6: Acto 3 — duelo en fase Sol



Figura 7: Acto 3 — fase Luna (cielo rojo) tras el flip

2.6 Interfaces de estado y HUD

El HUD y los paneles de estado se superponen en lienzos (**Canvas**) dedicados y se gestionan de forma modular. Se incluyen los siguientes elementos principales:

- **Barra de vida del jugador (PlayerHUD)**: se suscribe a `Health.onDamaged` (valor normalizado 0..1) y anima el relleno con un degradado de color y un suavizado.
- **Barra de vida de Jammo**: activa durante los Actos 2 y 3, permite al jugador monitorizar de forma constante el estado de salud del compañero robot, ya que su muerte provoca automáticamente el Game Over.
- **Barras de vida del duelo 1vs1 (Hermanos)**: elementos de interfaz específicos del Acto 3 que muestran simultáneamente la salud del jugador y la del jefe (el doble) durante las fases del enfrentamiento final.
- **Barra de progreso de la estatua (StatueProgressBar, Acto 2)**: barra vertical con menisco que refleja `StatueRig.Normalized`, indicando el porcentaje de ensamblaje.
- **Aparición/ocultación del HUD (HudReveal)**: *slide + fade* idempotente, usado por los directores para revelar el HUD tras los prólogos y ocultarlo en las transiciones/pausa/muerte.
- **Interfaz de Ajustes (SettingsUI y CRTController)**: menú de configuración que permite gestionar las opciones de audio (enrutado por `AudioMixer`) y

vídeo (incluyendo pantalla completa y la activación del filtro CRT retro), con persistencia de preferencias en PlayerPrefs.

- **Tabla de clasificación / Leaderboard** (LeaderboardUI + LeaderboardEntryView): lista persistente de puntuaciones finales que permite consultar el desempeño global. Cuenta con detalles específicos de cada nivel, desplegables de forma interactiva en acordeón.
- **Diálogos** (DialogueManager): lienzo de diálogo con *fade* y efecto de máquina de escribir.



Figura 8: Panel de Ajustes

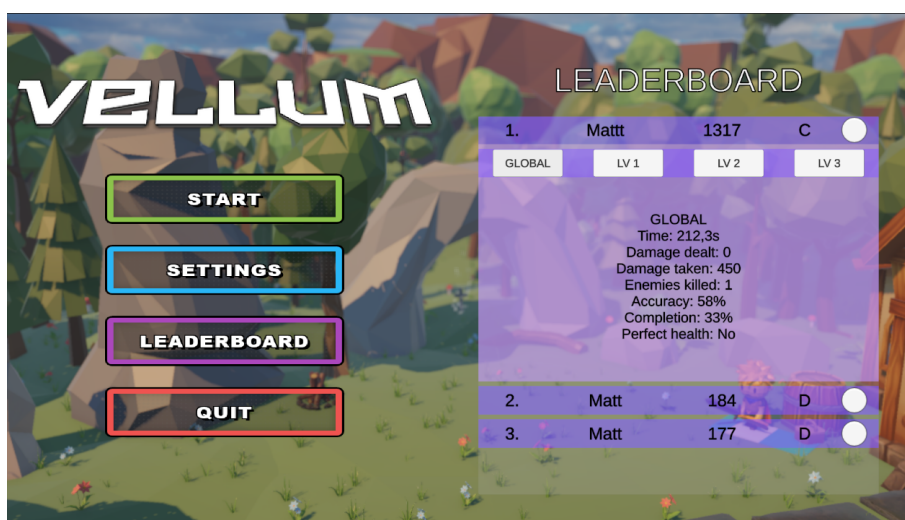


Figura 9: Interfaz de Tabla de clasificación (Leaderboard)

3 Algoritmos, Inteligencia Artificial y Comportamiento

Este capítulo es el **núcleo de ingeniería** del proyecto. Detalla la lógica que rige tanto el control del avatar del jugador como la autonomía de los adversarios, y profundiza en los **algoritmos diferenciados** de cada Acto: lógica difusa, generación procedimental, *steering* y máquinas de estados.

El control del **Jugador** es directo: traduce las entradas del usuario (nuevo Input System) en movimiento del `ThirdPersonController` de `StarterAssets` y en transiciones de animación (combo/finisher, defensa). El verdadero reto técnico está en la **IA de los enemigos**, que evita un controlador genérico único y emplea tres paradigmas distintos —**lógica difusa de Mamdani**, **máquina de estados finitos** y **árbol/heurística de búsqueda** para la generación de caminos—, además de un **pipeline de combate orientado a interfaces** que desacopla quién inflige daño de quién reacciona.

3.1 Controlador del Jugador

El controlador del jugador se reparte en componentes modulares: `PlayerCombat` (combate), `PlayerMeleeAttack` (detección de impacto), `PlayerHealth` (vida) y `PlayerSkinSwitcher` (cambio de modelo). La locomoción base se delega en el `ThirdPersonController` de `StarterAssets`.

- **Combo → Finisher (PlayerCombat)**. Los golpes ligeros se reproducen en una **capa enmascarada** del Animator (`UpperBody`): las piernas siguen en locomoción mientras el torso ataca. Tras `comboHitsBeforeFinisher` golpes, el siguiente clic lanza un **finisher de cuerpo completo** (la capa enmascarada baja su peso a 0 porque es un clip de la *Base Layer*). El peso de la capa sube de golpe al actuar y baja suavemente al volver a reposo. La defensa y el ataque son **mutuamente excluyentes**: levantar el escudo cancela el golpe “encarado”. El finisher es *comprometido*: bloquea el movimiento hasta terminar (vía `MeleeFinisherState`, un `StateMachineBehaviour` que avisa a `PlayerCombat` en *enter/exit*).
- **Detección de impacto (PlayerMeleeAttack)**. La detección **no** depende de eventos de animación, sino de un temporizador (*wind-up* + ventana activa): un `OverlapBox` hacia delante daña una vez por objetivo y por golpe a cual-

quier `IDamageable`. Expone `IsSwinging` (lo lee la IA del jefe para su defensa reactiva) y los eventos `Swung/HitLanded` (precisión para la puntuación).

- **Vida (`PlayerHealth`)**. Junto al componente compartido `Health`, al morir reproduce el clip de muerte en el `Animator` activo, espera y dispara `onPlayerDied` (enlazado a `ShowGameOver`).
- **Cambio de modelo (`PlayerSkinSwitcher`)**. *Hot-swap* del modelo (masculino/femenino) en tiempo de ejecución; tras el cambio **rebindea** el `Animator` (`Animator.Rebind()`) y notifica el evento `SkinChanged`, que el jefe final usa para reflejarse en el sexo opuesto.

3.2 Pipeline de combate orientado a datos

Todo el combate se construye sobre un pequeño conjunto de **interfaces** que desacoplan a quien ataca de quien reacciona, en lugar de referencias directas:

- `IDamageable.TakeDamage(DamageInfo)` — lo implementa cualquier entidad dañable.
- `DamageInfo` — estructura ligera con cantidad, posición de origen y `GameObject` atacante.
- `IDamageFilter.ShouldBlock(...)` — *gancho pre-daño*: si algún filtro devuelve `true`, el golpe se cancela (lo usa `FrontalShieldBlock` para el bloqueo frontal del jugador y `BossShield` para el del jefe).
- `IDamageReaction.OnDamaged(...)` — *gancho post-daño*: lo usa `KnockbackReceiver` (empuje del enemigo en arco) y `EnemyAI` (reaccionar al ser golpeado por el jugador).

El componente central es **Health**, compartido por jugador, enemigos, Jammo, jefe y estatua. Ejecuta el pipeline completo: ventana de invulnerabilidad → veto de filtros → aplicación del daño (con **suelo** y **tope por golpe** opcionales, usados por el duelo final) → reacciones → eventos de HUD/daño/muerte. Expone la salud normalizada (`Normalized`), `Heal`, `ResetHealth` (para pooling) y eventos `C#` (`Died`, `Damaged`) además de los `UnityEvent` del Inspector.

3.3 Motor de Inferencia Difusa Mamdani

El corazón de la IA es un **motor de lógica difusa de Mamdani** reutilizable, escrito desde cero en `AI/Fuzzy/`. Modela decisiones de los agentes con **reglas lingüísticas**

en lugar de cadenas de `if`. El ciclo de inferencia (`FuzzyController.Evaluate`) tiene cuatro pasos:

1. **Fuzzificación:** las entradas nítidas se evalúan en conjuntos lingüísticos (p. ej. *Near / Medium / Far*) mediante funciones de pertenencia (`MembershipFunction`: triángulo, trapecio y “hombros” izquierdo/derecho).
2. **Fuerza de regla:** = $\min$ de las pertenencias de los antecedentes (AND lógico).
3. **Agregación:** para cada salida, $\max$ de los conjuntos consecuentes recortados a la fuerza de su regla.
4. **Defuzzificación: centroide muestreado** sobre el dominio de salida.

El controlador se construye una sola vez con un **builder fluido** (`.Rule().If(...).And(...).Then(...)`), que resuelve las etiquetas a índices en tiempo de construcción. La evaluación es **sin asignaciones** (búferes de fuerzas preasignados; entradas y salidas como `float[]` alineados al orden de declaración). El motor es **compartido e inmutable**: cada agente solo guarda sus pequeños búferes de E/S.

Este mismo motor resuelve **tres problemas distintos**:

- **Agresión enemiga** (`EnemyFuzzyBrain`, Acto 2): entradas *distancia / vida / aglomeración* $\rightarrow$ salida *agresión*, que modula el *cooldown* de ataque (más agresión $\Rightarrow$ ataca más a menudo; en grupo se coordina el asalto).
- **Intención del jefe** (`BossFuzzyBrain`, Acto 3, fase Luna): dos controladores dedicados, uno decide si **interceptar** a un Jammo que transporta una pieza (entradas: distancia a Jammo + si transporta) y otro si **buscar un botín de vida** estando herido (entradas: vida propia + distancia al botín).
- **Calidad estética del camino** (`FuzzyPathEvaluator`, Acto 1): puntúa lo “bonito” que es un camino generado (densidad de giros, proximidad a los bordes, dispersión), para elegir el mejor entre varios candidatos.

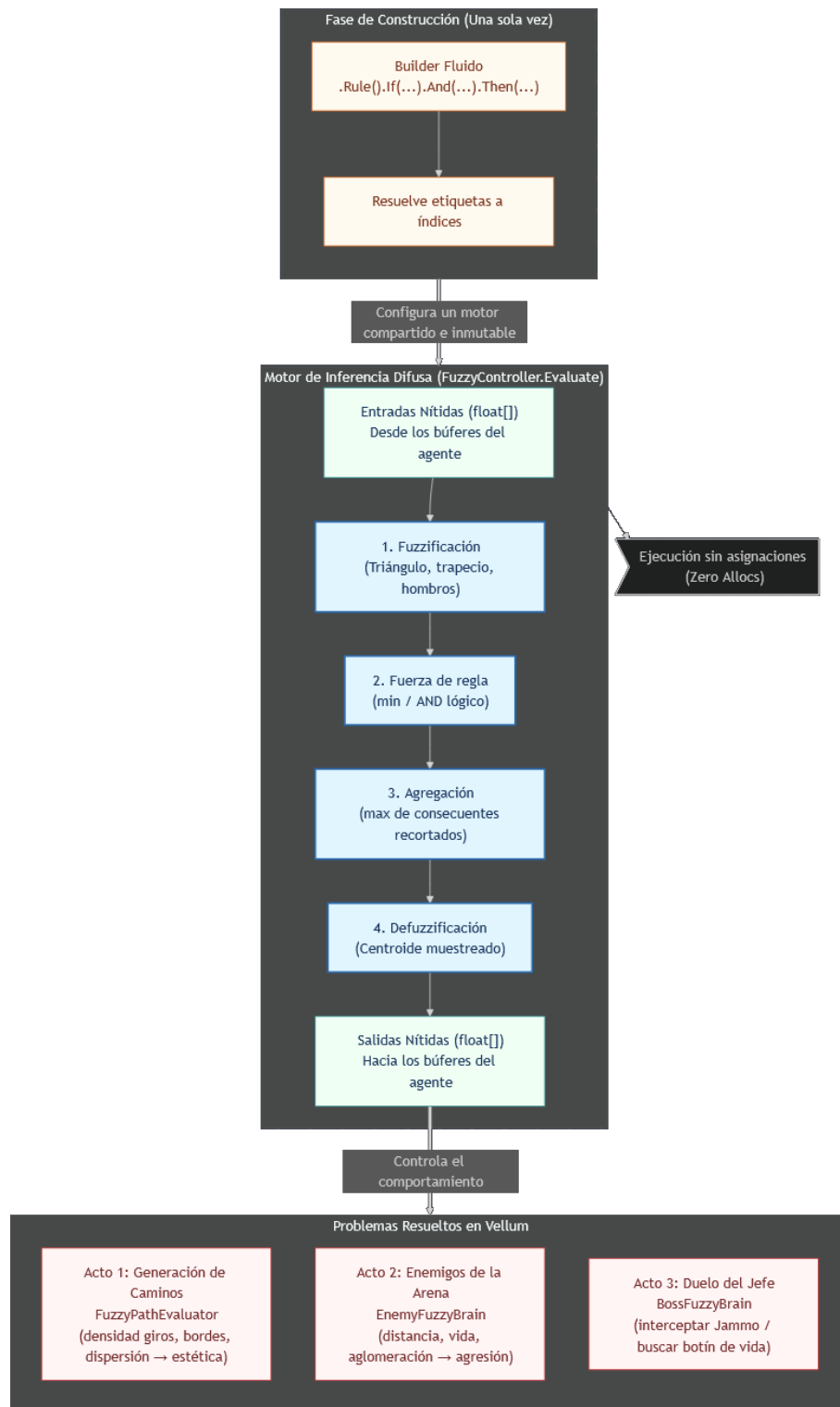


Figura 10: Esquema del ciclo Mamdani (fuzzificar → reglas → agregar → centroide)

3.4 Generación Procedimental de Caminos (Acto 1)

El camino del puzzle se genera con `SelfAvoidingPathGenerator`: produce un **camino auto-evitante** (cada celda usada una vez $\Rightarrow$ sin cruces por construcción) de **longitud exacta** (73 celdas) entre dos celdas, mediante **backtracking aleatorizado**. Las claves del algoritmo:

- **Poda Manhattan + paridad**: se descarta cualquier rama cuyos movimientos restantes no puedan alcanzar el objetivo con la **paridad** correcta (se valida la factibilidad antes de buscar y se poda en cada paso).
- **Heurística de Warnsdorff**: entre los movimientos legales se prefiere el vecino con **menos salidas libres** (con desempate tras una baraja aleatoria), lo que da convergencia rápida y caminos variados.
- **Espaciado de 8 vecinos** y regla **anti-zig-zag** (no dos giros consecutivos): mantienen los corredores paralelos separados y el camino legible.
- **Penúltima baldosa obligada**: la celda penúltima está fijada, de modo que la dirección de llegada a la baldosa final es siempre la misma.
- **Selección multi-candidato**: se generan varios candidatos y se conserva el de **mayor puntuación estética difusa** (`FuzzyPathEvaluator`).
- **Presupuesto de nodos y reinicios**: cada intento tiene un tope de nodos expandidos; si se agota, reinicia con un nuevo orden aleatorio (evita ramas patológicas).

El resultado es un puzzle **distinto en cada partida pero consistentemente bien formado**. Si las restricciones (Manhattan/paridad de inicio/fin/penúltima) son infeasibles, el generador lanza un error descriptivo por diseño.

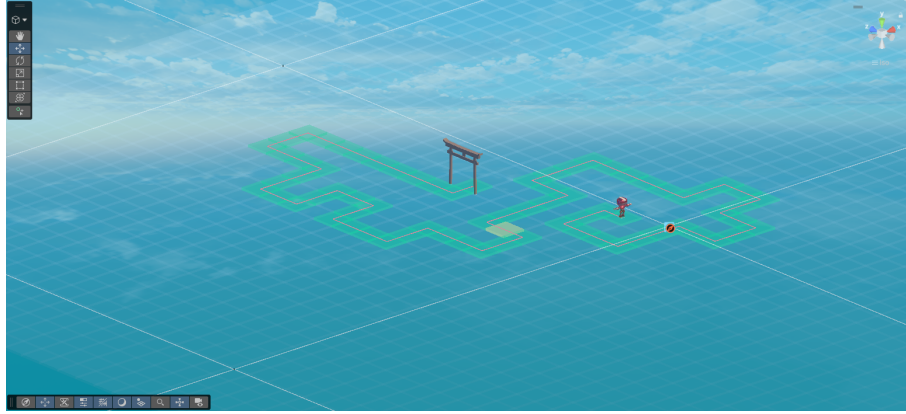


Figura 11: Ejemplo de camino generado sobre la rejilla (gizmo verde + checkpoint amarillo)

3.5 Steering y Cerco Angular (Acto 2)

Los enemigos de la arena (**EnemyAI**) se mueven **sin NavMesh**, con *steering* ligero:

- **Separación tipo boids:** cada enemigo se aparta de sus vecinos con peso inverso al cuadrado de la distancia (`Physics.OverlapSphereNonAlloc`, sin asignaciones por frame).
- **Cerco en “abanico”:** un coordinador compartido (`EnemyTargetCoordinator`) asigna a cada atacante una **ranura angular** alrededor del objetivo, de modo que se reparten en lugar de apilarse en fila. Los que llegan por detrás apuntan a una ranura libre y rodean.
- **Objetivo dinámico:** por defecto persiguen a **Jammo** (sabotean la estatua); un enemigo cambia al **jugador** solo si este lo golpea (reacción `IDamageReaction.OnDamaged`) y hay una ranura de “perseguidor del jugador” libre (limitadas por `maxPlayerChasers`).
- **Parada segura:** la distancia de parada nunca baja de la suma de los radios reales de las cápsulas (no empuja el colisionador del objetivo).
- **Agresión difusa:** el ritmo de ataque lo fija `EnemyFuzzyBrain` (§3.3).

Los enemigos se **reciclan con pooling** (`ResetState/Configure`) en lugar de crearse y destruirse.



Figura 12: Oleada de enemigos rodeando a Jammo en abanico

3.6 Máquina de Estados del Jefe + Defensa Reactiva (Acto 3)

El jefe final (BossDuelAI) usa una **máquina de estados finitos con ventanas temporales** para variar el ritmo del combate, separada de EnemyAI (aquí hay un único jefe, sin coordinador). La locomoción es por CharacterController (sin NavMesh), con un *blend tree* andar/correr alimentado con la velocidad real en m/s.

- **Estados:** **Aggro** (persigue y, en rango, ejecuta un **combo** de varios golpes estando quieto), **Reposition** (corre a un punto aleatorio cercano), **Guard** (mantiene la guardia y espera, luego decide si atacar o reposicionar), más **Kite** y **SeekHealth** en la fase Luna potenciada.
- **Defensa reactiva:** por encima de la FSM, cuando detecta el **inicio del golpe del jugador** (PlayerMeleeAttack.IsSwinging) a corta distancia y de frente, levanta la guardia de forma **probabilística** para bloquear ese golpe concreto.
- **Fase Luna (potenciada):** más velocidad y daño; puede **romper el combate para interceptar a Jammo** según la decisión difusa de BossFuzzyBrain (con **histéresis** para no oscilar) y puede ir a **curarse** a un botón neutral (DuelHealthSpawner) cuando está herido. El director pausa al jefe durante los *flips* y la ventana del golpe final (SetPaused).

El **escudo** del jefe (BossShield) cumple dos papeles como IDamageFilter: **bloqueo frontal** cuando defiende y **conteo de golpes** durante la recolección (cada hitsPerPiece aciertos avisa al director para desbloquear una pieza).

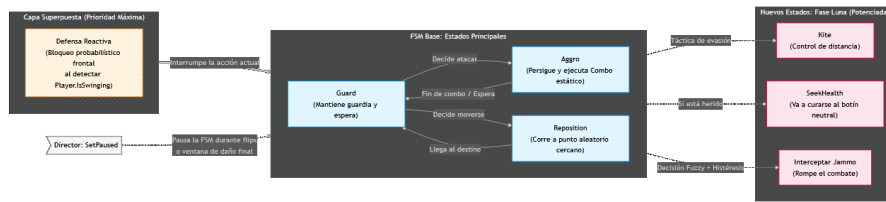


Figura 13: Diagrama de estados del jefe (Aggro / Reposition / Guard / Kite / SeekHealth)

3.7 El compañero Jammo

Jammo cumple un papel mecánico distinto en cada Acto, reutilizando patrones de movimiento:

- **Acto 1 (guía):** JammoCompanion lo activa una vez (diálogo + animación de “levantarse” + inicio del puzzle) y JammoGuideController lo hace **recorrer el camino correcto** iluminando el rastro, mientras el jugador queda bloqueado.
- **Actos 2 y 3 (porteador):** JammoCarrier lo controla con un NavMeshAgent para esquivar obstáculos: camina a un punto, **recoge una pieza** que flota a su lado y la **deposita**. Es **vulnerable** (IDamageReaction): si lo golpean mientras transporta, marca la pérdida de la pieza. Su vida (JammoHealth) provoca **Game Over** si llega a 0.

3.8 Cinemáticas, cámaras y “flip” del mundo

Las cinemáticas siguen un patrón fijo del proyecto: **bloquear input** → **manejar prioridades de Cinemachine 3** → **secuencia temporizada por corrutina** → **restaurar estado**.

- **CinematicFallManager:** caída cinemática inicial (lock → cámara cenital → corrutina que conduce un **shader de disolución global** vía `Shader.SetGlobalVector/Float` → restaurar). Su método `SetPlayerMovement(canMove, keepLookActive)` es el bloqueo de jugador reutilizado por varios sistemas.
- **Act02Director / MirrorDuelDirector:** orquestan prólogo/combate/epílogo del Acto 2 y todo el duelo del Acto 3 (prólogo, fases, transición, epílogo) usando *blends* lentos de Cinemachine, diálogos y caminatas animadas de los actores.

- **MirrorFlipDirector + MirrorCameraSync:** la transición **Sol↔Luna**. En vez de hacer *crossfade* de dos skyboxes (imposible sin un *blend shader*), **interpola** las propiedades de un material procedural de cielo en tiempo de ejecución, hace *crossfade* de luces, rota los cuerpos celestes y **cambia las capas UpWorld/DownWorld**; el cambio queda enmascarado por la rotación. MirrorCameraSync refleja la cámara sobre el plano de agua.
- **Comunicación global con shaders:** mediante `Shader.SetGlobalVector` / `SetGlobalFloat` (URP Shader Graph) en lugar de referencias por material —p. ej. la disolución gris de victoria del Acto 3—, conforme a las convenciones de rendimiento del proyecto.

4 Arquitectura del Código

Esta sección documenta de forma estructurada los componentes lógicos del proyecto, organizados según la jerarquía de carpetas de `Assets/Scripts/`. Para cada script se detallan su propósito, sus **parámetros configurables** (`[SerializeField]` / públicos relevantes) y la responsabilidad de sus **métodos principales**. Todo el código está documentado en inglés con `/// <summary>` y `[Tooltip]`, por lo que se mantienen los nombres originales.

4.1 Carpeta: Player

4.1.1 `PlayerCombat.cs`

Conduce el combate cuerpo a cuerpo del jugador: combo de golpes ligeros sobre una capa enmascarada del Animator que escala a un *finisher* de cuerpo completo, más un estado de defensa/escudo. Ataque y defensa son mutuamente excluyentes; el movimiento se bloquea durante las acciones comprometidas.

Parámetros:

- `moveWhileActing` (bool): permite moverse durante ataque/defensa (requiere capa superior con máscara, si no las piernas resbalan).
- `upperBodyLayerName` (string): nombre de la capa enmascarada de golpes ligeros.
- `comboHitsBeforeFinisher` (int): golpes enmascarados antes de que el siguiente clic lance el *finisher*.
- `upperBodyBlendSpeed` (float): velocidad (peso/seg) con la que la capa superior vuelve a 0.
- `IsDefending` (bool, propiedad): si el jugador está defendiendo (la leen `FrontalShieldBlock`, `PlayerMovements`, etc.).

Métodos principales:

- `RefreshAnimator(GameObject)`: lo llama `PlayerSkinSwitcher` tras cambiar de modelo, para reapuntar el Animator cacheado, recalcular el índice de capa y resetear el combo.
- `Update()`: gestiona la entrada (ratón), el peso dinámico de la capa superior, el reset del combo y el bloqueo de movimiento durante acciones comprometidas.

- `OnFinisherStart()` / `OnFinisherEnd()`: invocados por `MeleeFinisherState` (SMB) al entrar/salir del finisher; ponen a 0 el peso de la capa y limpian el flag/combo.

4.1.2 `PlayerMeleeAttack.cs`

Detección de impacto del golpe **dirigida por input** (no por Animation Event): idéntica para ambos modelos. `PlayerCombat` llama a `BeginSwing()` en cada clic.

Parámetros:

- `attackDamage` (float): daño base.
- `windupTime`, `activeTime` (float): retardo antes de activar la *hitbox* y duración de la ventana activa.
- `hitboxHalfExtents` (Vector3), `forwardOffset`, `verticalOffset` (float): geometría de la *hitbox* relativa al jugador.
- `hittableLayers` (LayerMask): capas dañables.
- `IsSwinging` (bool, propiedad): activo durante el *swing* (lo lee el jefe para su defensa reactiva).
- Eventos `Swung` / `HitLanded`: denominador/numerador de la precisión para la puntuación.

Métodos principales:

- `BeginSwing(float)` / `CancelSwing()`: inician/cancelan el *swing* (cancelar evita el golpe “encarado” tras defender).
- `SwingRoutine()` (coroutine): *wind-up* + ventana activa aplicando golpes cada frame.
- `ApplyHit()`: `OverlapBox` hacia delante; daña una vez por objetivo y por *swing* a cada `IDamageable`.

4.1.3 `MeleeFinisherState.cs`

`StateMachineBehaviour` colocado en el estado `Attack04` (finisher de cuerpo completo) de la *Base Layer*. En `OnStateEnter/OnStateExit` notifica a `PlayerCombat` para que ponga a 0 el peso de la capa superior y bloquee el movimiento durante el giro. En el jefe (clon sin `PlayerCombat`) las llamadas son inocuas.

4.1.4 PlayerHealth.cs

Acompaña a `Health` en el jugador. Al morir, gestiona la secuencia de muerte: reproduce el clip de animación en el `Animator` activo, espera un tiempo determinado y dispara el evento de fin de juego.

Parámetros:

- `onPlayerDied (UnityEvent)`: evento invocado *después* de la animación de muerte. Se enlaza desde el Inspector a la lógica de fin de juego (por ejemplo, `MainMenuManager.ShowGameOver`).
- `deathTriggerName (string)`: nombre del *trigger* de muerte. Ambos controladores del `Animator` (modelo masculino y femenino) deben usarlo de forma idéntica.
- `deathAnimationDuration (float)`: tiempo en segundos asignado para mostrar el clip de muerte antes de invocar el evento y mostrar la pantalla de Game Over.

Propiedades:

- `Health`: expone de forma pública el componente `Health` asociado al jugador.

Métodos principales:

- `HandleDied()`: método público enlazado a `Health.onDied` desde el Inspector. Utiliza un *flag* interno (`_isDying`) para asegurar que la secuencia arranque una sola vez, iniciando la corrutina `DeathRoutine()`.
- `DeathRoutine()`: corrutina privada que ejecuta la secuencia completa. Primero, bloquea el movimiento y el combate del jugador (aprovechando `DialogueManager.LockPlayer()`); luego busca el `Animator` de la geometría activa, valida la existencia del parámetro con `AnimatorParameterCache` y dispara el *trigger* de muerte. Finalmente, espera `deathAnimationDuration` segundos en tiempo real y ejecuta `onPlayerDied.Invoke()`.

4.1.5 PlayerSkinSwitcher.cs

Gestiona el *hot-swap* (intercambio en caliente) del modelo 3D del jugador (masculino/femenino) en tiempo de ejecución. Al cambiar de aspecto, se encarga de reasignar el `Animator` activo a los controladores de movimiento y combate, y notifica a otros sistemas.

Parámetros:

- `maleGeometry / femaleGeometry (GameObject)`: referencias a las mallas y *rigs* de ambos modelos en la jerarquía del jugador.

Propiedades y Eventos:

- `IsMale (bool)`: propiedad pública (de solo lectura) que indica si la forma actual es la masculina. Es leída por el jefe final (`EnemySkinMirror`) para adoptar siempre la forma opuesta.
- `SkinChanged (Action<bool>)`: evento lanzado cada vez que ocurre un cambio de piel, pasando el estado actual de `IsMale`.

Métodos principales:

- `SwitchToMale() / SwitchToFemale()`: métodos públicos que sirven como envoltorios (*wrappers*) para invocar la lógica interna de cambio. Por defecto, en el `Start()`, se inicializa con el modelo masculino.
- `Switch(bool male)`: método privado que ejecuta la lógica central en tres pasos:
 1. Activa el `GameObject` de la geometría seleccionada y desactiva la opuesta (`SetActive`).
 2. Notifica a los scripts dependientes (`ThirdPersonController.RebindAnimator()` y `PlayerCombat.RefreshAnimator()`) para que actualicen sus referencias internas hacia el nuevo `Animator` recién activado.
 3. Dispara el evento `SkinChanged` para notificar a los suscriptores (como el director o el jefe final).

4.1.6 AnimationEventForwarder.cs

Va en el hijo con el `Animator` y **reenvía** los eventos de animación de `StarterAssets` (`OnFootstep`, `OnLand`) hacia el `ThirdPersonController` del padre, ya que esos eventos disparan en el objeto dueño del `Animator`.

4.2 Carpeta: Combat

4.2.1 Health.cs

Componente de vida **compartido** por todos los actores de combate (jugador, enemigos, `Jammo`, jefe, estatua). Implementa la interfaz `IDamageable` y ejecuta el

pipeline de daño completo, desacoplando la lógica de combate de la de visualización mediante el uso intensivo de eventos y delegados.

Parámetros de Inspector:

- `maxHealth` (float): la cantidad máxima de vida de la entidad.
- `invulnerabilityDuration` (float): ventana de invulnerabilidad tras recibir un impacto. Varía según la entidad (ej. jugador $\sim 0.5s$, enemigos $\sim 0.1s$, estatua $0s$).
- `onDamaged` (`HealthChangedEvent`): `UnityEvent` que transporta el valor de salud normalizado ($0..1$), utilizado principalmente para actualizar la interfaz (HUD) tanto al recibir daño como al curarse.
- `onDied` (`UnityEvent`): invocado exactamente una vez cuando la salud llega a 0, utilizado para enlazar animaciones o interfaces de final de juego desde el Inspector.

Propiedades y Eventos en tiempo de ejecución (C#):

- **Propiedades:** `CurrentHealth` (vida actual), `IsDead` (estado de muerte), `MaxHealth` y `Normalized` (valor entre 0 y 1).
- `Died` (`Action`): evento delegado que se invoca de forma simultánea a `onDied`. Permite a los sistemas dinámicos (como `WaveManager` para el pooling) suscribirse sin necesidad de cableado en el Inspector.
- `Damaged` (`Action<DamageInfo>`): evento disparado **exclusivamente** al recibir daño (a diferencia de `onDamaged`, que también se dispara al curarse). Se utiliza en el duelo final para generar botines de vida (pickups) con cada golpe exitoso, sin caer en un bucle infinito al curar.

Métodos principales (El Pipeline de Daño):

- `TakeDamage(DamageInfo info)`: ejecuta la secuencia completa de procesamiento de daño:
 1. Verifica si la entidad ya está muerta o si se encuentra dentro de la **ventana de invulnerabilidad**.
 2. Interroga a todos los componentes `IDamageFilter` asociados al objeto (*vetoes pre-daño*). Si alguno devuelve `true` (ej. bloqueo con escudo), el daño se cancela.

3. Aplica topes opcionales de daño (`_maxDamagePerHit`) y un suelo de daño (`_damageFloor`) que impide que la vida baje de cierto límite (utilizado para controlar las fases del jefe final).
 4. Notifica a todos los componentes `IDamageReaction` asociados (como animaciones de *knockback*).
 5. Finalmente, invoca los eventos pertinentes (`onDamaged`, `Damaged` y, si la vida llega a 0, `onDied` y `Died`).
- `Heal(float amount)`: restaura la vida sin exceder el máximo. Si la entidad está muerta, no la revive. Tras curar, invoca `onDamaged` para refrescar el HUD.
 - `ResetHealth()`: restablece la salud al máximo y limpia los *flags* de estado (muerte, *cooldown* de golpes y topes), preparándolo para el *pooling* o el reinicio de la arena.
 - `SetDamageFloor(float)` y `SetMaxDamagePerHit(float)`: reguladores en tiempo de ejecución que el director del Acto 3 inyecta en el jefe final para las reglas del duelo.

4.2.2 DamageInfo.cs

`struct` ligero que viaja por el pipeline: `amount` (`float`), `sourcePosition` (`Vector3`, para escudo frontal y *knockback*) y `source` (`GameObject` atacante).

4.2.3 IDamageable.cs

Interfaz de lo dañable: `void TakeDamage(DamageInfo info)`.

4.2.4 IDamageFilter.cs

Gancho **pre-daño**: `bool ShouldBlock(DamageInfo info)` (si devuelve `true`, el golpe se cancela).

4.2.5 IDamageReaction.cs

Gancho **post-daño**: `void OnDamaged(DamageInfo info)`.

4.2.6 KnockbackReceiver.cs

Componente que se adjunta **exclusivamente** a los enemigos (nunca al jugador). Implementa la interfaz `IDamageReaction` para generar una respuesta física simulada

al recibir daño: un empuje en arco (retroceso combinado con un pequeño salto) al estilo *Minecraft*.

Parámetros de Inspector:

- **horizontalSpeed** y **upwardSpeed** (**float**): definen las velocidades iniciales de desplazamiento y elevación del arco.
- **knockbackGravity** (**float**): gravedad personalizada (generalmente muy alta, ej. -22f) aplicada exclusivamente durante el salto del impacto.
- **maxAirTime** (**float**): medida de seguridad. Fuerza la terminación del empuje si el enemigo no toca el suelo tras este tiempo (previene cuelgues en terrenos irregulares o abismos).
- **knockbackCooldown** (**float**): tiempo mínimo entre dos empujes. Evita que una ráfaga de golpes rápidos (clics encadenados) lance al enemigo excesivamente lejos.

Propiedades:

- **IsKnockbackActive** (**bool**): indica si la secuencia de retroceso se está reproduciendo actualmente (basado en si la corrutina es nula o no).

Métodos principales:

- **OnDamaged(DamageInfo info)**: implementación del gancho *post-daño*. Verifica que no haya un empuje activo y que se haya superado el *cooldown*. Luego, calcula el vector de dirección horizontal opuesto al origen del ataque (**info.sourcePosition**), lo normaliza y arranca la corrutina.
- **KnockbackArcRoutine(Vector3 horizontalDir)**: corrutina privada que integra el movimiento en cada *frame*. Aplica la aceleración de la gravedad a la velocidad vertical y desplaza al enemigo usando **CharacterController.Move()**. El bucle termina cuando la entidad se encuentra en fase descendente (**vY < 0**) y vuelve a tocar un suelo real (**_controller.isGrounded**), o si se excede el **maxAirTime**.
- **CancelKnockback()**: interrumpe inmediatamente cualquier empuje activo deteniendo la corrutina en seco, restableciendo el estado (útil al morir o al ser desactivado por el *pool*).

4.2.7 FrontalShieldBlock.cs

En el jugador, junto a `Health`. Como `IDamageFilter`, cancela el daño entrante cuando el escudo está alzado (`PlayerCombat.IsDefending`) y el atacante cae en el cono frontal.

Parámetros: `blockAngle` (float); evento `Blocked` (para la puntuación de bloques).

Método: `ShouldBlock(DamageInfo)` (comprueba defensa + cono frontal).

4.3 Carpeta: AI/Fuzzy

4.3.1 FuzzyController.cs

Es el núcleo del motor de inferencia **Mamdani** propietario. Está diseñado para ser altamente eficiente: se construye una sola vez utilizando un patrón **builder fluido** y, a partir de ese momento, la evaluación del ciclo de inferencia funciona **sin asignaciones de memoria** dinámica (allocation-free), ideal para la ejecución continua (frame a frame) en Unity.

El Ciclo de Evaluación (Evaluate): El método `Evaluate(float[] crispInputs, float[] crispOutputs)` recibe búferes preasignados alineados con el orden de declaración de las variables y ejecuta tres pasos matemáticos:

1. **Fuzzificación y Fuerza de Regla:** Para cada regla, calcula la pertenencia de cada entrada a su conjunto difuso y determina la fuerza total de la regla aplicando un operador `min` (AND lógico) sobre todos los antecedentes. Almacena estos resultados en un arreglo interno `_strengths`.
2. **Agregación:** Para cada variable de salida, itera sobre un número definido de muestras discretas (`_samples`) a lo largo de su dominio. Para cada muestra x , encuentra el valor máximo (`max`) entre los consecuentes de las reglas aplicables, siempre recortando (clipping) el conjunto consecuente a la fuerza de su respectiva regla.
3. **Defuzzificación:** Utiliza la técnica del **centroide** (centro de gravedad) sobre las muestras agregadas ($(\sum(x \cdot \text{agg}))/\sum \text{agg}$) para obtener y devolver un valor nítido (crisp) en el arreglo `crispOutputs`.

El Builder Anidado (Builder): Permite definir el controlador usando un lenguaje natural y fluido en C#.

- **API de configuración:** `.Samples(n)` define la resolución del centroide; `.Input(v)` y `.Output(v)` registran las variables.

- **Sintaxis de reglas (RuleDraft):** Permite encadenar llamadas como `.Rule().If(var, set).And(var, set).Then(var, set)`.
- **Resolución en tiempo de construcción:** Al invocar `.Build()`, el **builder** convierte todos los nombres de variables y etiquetas de conjuntos en **índices enteros**. El controlador final ignora por completo las cadenas de texto (strings) durante la ejecución, optimizando drásticamente el rendimiento.

Fragmento clave del código de Evaluación:

```

1  // 2)+3) for each output, centroid of the aggregate (max of the clipped sets).
2  for (int o = 0; o < _outputs.Length; o++)
3  {
4      FuzzyVariable outVar = _outputs[o];
5      float min = outVar.Min, max = outVar.Max;
6      float step = (max - min) / (_samples - 1);
7
8      float num = 0f, den = 0f;
9      for (int k = 0; k < _samples; k++)
10     {
11         float x = min + step * k;
12         float agg = 0f;
13         for (int r = 0; r < _rules.Length; r++)
14         {
15             if (_rules[r].OutputVar != o || _strengths[r] <= 0f) continue;
16             // Clipping: el consecuente se recorta a la fuerza de la regla
17             float clipped = Mathf.Min(_strengths[r], outVar.Membership(_rules[r].OutputSet, x));
18             // Aggregation: el maximo de los conjuntos recortados
19             if (clipped > agg) agg = clipped;
20         }
21         num += x * agg;
22         den += agg;
23     }
24     crispOutputs[o] = den > 1e-6f ? num / den : (min + max) * 0.5f;
25 }

```

Listing 1: Extracto de la defuzzificación por centroide en `FuzzyController.Evaluate`

4.3.2 FuzzyVariable.cs

Variable lingüística: nombre, dominio [Min,Max] y conjuntos difusos etiquetados.

API: `Set(label, MembershipFunction)` (fluido), `IndexOf(label)`, `Membership(setIndex, x)`, `SetCount`.

4.3.3 FuzzyRule.cs

Regla resuelta a **índices** (sin cadenas en runtime): `AntecedentVar[]`, `AntecedentSet[]`, `OutputVar`, `OutputSet`. `readonly struct` immutable.

4.3.4 MembershipFunction.cs

Función de pertenencia (`readonly struct`, sin asignaciones). Factorías: `Triangle(a,b,c)`, `Trapezoid(a,b,c,d)`, `LeftShoulder(a,b)` (“bajo/pocos”), `RightShoulder(a,b)` (“alto/muchos”); `Evaluate(x)` devuelve la pertenencia en $[0,1]$.

4.4 Carpeta: Enemies

4.4.1 EnemyAI.cs

Es el núcleo de inteligencia artificial de los enemigos de la arena (Acto 2). Opera **sin NavMesh**, basándose en un sistema de *steering* (dirección) personalizado. Combina la asignación dinámica de objetivos (vía `EnemyTargetCoordinator`), la modulación del ritmo de ataque mediante lógica difusa (`EnemyFuzzyBrain`) y técnicas de separación tipo *boids* para evitar aglomeraciones, logrando un cerco en abanico alrededor del objetivo.

Parámetros de Inspector:

- **Movimiento:** `moveSpeed`, `turnSpeed` y `gravity` rigen la locomoción a través del `CharacterController`.
- **Separación (Anti-solapamiento):** `enemyMask` (capa para detectar a otros enemigos), `separationRadius` y `separationWeight` (fuerza de repulsión entre entidades), y `peerCheckRadius` (radio para contar aliados y alimentar la entrada *crowding* del cerebro difuso).
- **Ataque y Contacto:** `attackRange` (distancia de impacto), `stopDistance` (distancia deseada para detenerse y rodear) y `collisionMargin` (margen de seguridad añadido a los radios de los colisionadores para evitar que el enemigo empuje físicamente al objetivo). Los tiempos `maxAttackCooldown` y `minAttackCooldown` definen los extremos del ritmo de ataque.
- **Cerco:** `fanSpacingDeg` define los grados de separación angular entre los atacantes alrededor de una misma víctima.
- **Frecuencias de actualización:** `decisionInterval` y `retargetInterval` evitan evaluar la lógica difusa y la búsqueda de objetivos en cada *frame*, optimizando el rendimiento.

Métodos principales y Lógica de comportamiento:

- `Update()`: bucle principal de la IA. Gestiona la muerte, pausa la lógica si hay un *knockback* activo, actualiza los temporizadores de decisión y reevalúa el objetivo. Calcula el punto de aproximación, orienta al enemigo, gestiona el ataque si está en rango y aplica el vector de separación para el movimiento.
- `ResolveTarget()`: interroga al `EnemyTargetCoordinator`. Persigue a Jammo por defecto, pero si el enemigo ha sido golpeado por el jugador (y hay cupo libre), fija su atención en este último.
- `ComputeApproachPoint(Vector3, Vector3)`: en lugar de caminar directamente hacia el centro del objetivo, calcula un punto de espera situado en la ranura angular (*slot*) asignada por el coordinador. Esto distribuye a los atacantes en forma de abanico.
- `TickFuzzy(float)` y `CountNearbyAllies()`: alimenta el controlador Mamdani con la distancia, la vida propia y el número de aliados cercanos (usando `OverlapSphereNonAlloc` sin asignaciones de memoria) para obtener un índice de agresión.
- `ComputeSeparation()`: calcula un vector de repulsión respecto a los compañeros cercanos (inversamente proporcional al cuadrado de la distancia) para mantener las formaciones orgánicas.
- `TryAttack(Transform)`: ejecuta el golpe. El tiempo de recarga (*cooldown*) exacto se interpola entre los valores máximo y mínimo utilizando la **agresión difusa** resultante del `TickFuzzy`, añadiendo una ligera aleatoriedad para evitar que todos los enemigos ataquen en el mismo frame exacto.
- `OnDamaged(DamageInfo)`: implementación de `IDamageReaction`. Si el daño proviene del jugador, activa un *flag* interno (`_triggeredByPlayer`) que forzará un cambio de objetivo en la próxima evaluación de `ResolveTarget()`.
- `ResetState()` / `Configure()`: métodos fundamentales para el **pooling**. Restablecen los temporizadores, la velocidad, el *TargetCoordinator* y fuerzan un `Rebind()` del Animator al reusar la entidad muerta para una nueva oleada.

4.4.2 EnemyFuzzyBrain.cs

Controlador difuso de los enemigos (compartido, inmutable). `Decide(EnemyPerception)` → `EnemyDecision` con **aggression**. Define las variables *distance* / *health* / *crowding* → *aggression* y la base de reglas (p. ej. *Near* + *High health* ⇒ agresión *High*;

Near + Low health $\Rightarrow$ *Low*; *Crowding High + Near* $\Rightarrow$ *High*). Incluye los **struct** auxiliares **EnemyPerception** y **EnemyDecision**.

4.4.3 **EnemyTargetCoordinator.cs**

Un *Singleton* ligero que actúa como árbitro o coordinador táctico en la arena del Acto 2. Permite que los enemigos (que se reciclan constantemente mediante *pooling*) se registren y obtengan sus objetivos sin necesidad de referencias inyectadas por instancia. Su función central es dictar las reglas de enfrentamiento: el objetivo predeterminado siempre es sabotear a Jammo.

Parámetros de Inspector:

- **player / playerHealth (Transform, Health)**: referencias directas al jugador.
- **jammo / jammoHealth (Transform, Health)**: referencias directas al compañero robot.
- **maxPlayerChasers (int)**: límite estricto de enemigos que pueden concentrarse en el jugador simultáneamente. Una vez alcanzado, el resto de la oleada se verá obligado a perseguir a Jammo, evitando abrumar al jugador.

Estado Interno (El Cerco en Abanico):

- Mantiene dos listas privadas **ordenadas** (**_playerChasers** y **_jammoChasers**).
- **Diseño clave**: el índice (posición) que ocupa un enemigo dentro de su respectiva lista define su “ranura” (*slot*). Este índice es leído por **EnemyAI** para rotar su vector de acercamiento, logrando orgánicamente la formación en abanico sin colisiones.

Métodos principales (API pública):

- **TryClaimPlayer(EnemyAI)**: evalúa si un enemigo que acaba de ser golpeado por el jugador puede cambiar su objetivo hacia él. Devuelve **true** si hay cupo libre (moviendo al enemigo a la lista del jugador) o **false** si se llegó al tope de **maxPlayerChasers**.
- **RegisterJammo(EnemyAI)**: asigna al enemigo a la lista de persecución de Jammo (y lo retira de la del jugador si estaba allí).
- **Unregister(EnemyAI)**: elimina al enemigo de todas las listas, liberando su ranura angular. Llamado obligatoriamente al morir o desactivarse por el *pool*.

- `SlotIndex(EnemyAI, isPlayer)` y `AttackerCount(isPlayer)`: métodos de consulta utilizados por `EnemyAI.ComputeApproachPoint()` para saber cuántos aliados comparten el mismo objetivo y qué ángulo exacto le corresponde a cada uno.
- `JammoAlive` / `PlayerAlive`: propiedades de validación rápida para evitar que los enemigos persigan referencias muertas o nulas.

4.5 Carpeta: `Level_1` (Acto 1 — Puzzle del Camino)

4.5.1 `PathPuzzleManager.cs`

Orquesta de forma centralizada toda la lógica del Acto 1. Tras generar el camino procedimental, coordina la caminata-guía de Jammo, evalúa en tiempo real las baldosas que pisa el jugador, revela gradualmente la puerta de salida y gestiona una pista de memoria limitada, así como las rutinas de penalización/reaparición si el jugador se equivoca.

Parámetros de Inspector (Selección):

- **Generación y anclajes:** `startTile`, `endTile` y `endApproachTile` fijan los puntos cardinales del camino en el mundo. `tileSize`, `randomSeed` (si es distinto a 0 fuerza un camino determinista) y `candidateCount` afinan la generación.
- **Entorno y referencias:** `globalFloor` (cuyos colisionadores se desactivan dinámicamente al empezar el puzzle para habilitar la caída), y `failResetDelay` (tiempo de caída antes de reaparecer).
- **Pista de memoria:** `memoryHintCharges` (cargas máximas por intento), `hintRevealCount` (baldosas a iluminar hacia delante), y el grupo `hintRevealDuration` / `hintBlinkCount` / `hintBlinkInterval` (tiempos de iluminación fija y parpadeo final).
- **Progreso y Puerta:** `doorController` (para enviar el parámetro de altura al *shader*), dividido en pasos escalonados (`doorTotalSteps`, `doorTilesPerStep`).
- **Eventos de HUD y Score:** Múltiples `UnityEvent` (`onPuzzleStarted`, `onCheckpointReached`, `onFail`, `onCorrectStep`) que enlazan la jugabilidad de este acto con el sistema global de puntuación (`ScoreManager`).

Métodos principales y Lógica:

- **GenerateRandomPath()**: llamado al iniciar la escena o desde el Editor. Evalúa la distancia de *Manhattan* entre inicio y fin comprobando si la paridad es compatible con los 73 pasos exigidos. Luego, invoca al generador procedimental de caminos y vuelca las coordenadas de la rejilla a referencias reales **PathTile**. Calcula automáticamente el índice del *checkpoint* (mitad exacta).
- **StartPuzzleSequence()**: desactiva el suelo global (buscando cuidadosamente y desactivando **solo** los colisionadores de base, respetando los de las baldosas), inicializa los contadores de la puerta e instruye a Jammo para caminar hasta el *checkpoint*.
- **CheckPlayerStep(PathTile)**: el corazón de la validación. Implementa tres reglas lógicas:
 1. **Exploración libre / Retroceso (Backtracking)**: si se pisa una baldosa ya validada previamente, solo se ilumina y se permite (no avanza el progreso de la puerta).
 2. **Avance correcto**: si es exactamente la siguiente baldosa en la secuencia, avanza el índice. Si se está bajo el **Período de Gracia** (**_inGracePeriod**), se permiten “saltos de hito” (ej. si el jugador entra directamente en el *checkpoint* tras la guía de Jammo).
 3. **Fallo**: si se pisa una baldosa errónea fuera del período de gracia, se le retira la solidez física provocando la caída del jugador y arrancando **FailAndReset()**.
- **FailAndReset()**: corrutina que detiene cualquier pista de memoria activa, espera, y luego **teletransporta** al jugador al punto de inicio. Para hacerlo con éxito en Unity, desactiva temporalmente el **CharacterController** durante un *frame*. Finalmente, re-ilumina de azul el rastro ya descubierto para que el jugador no tenga que memorizarlo de cero.
- **MemoryHintRoutine()**: corrutina de pista. Bloquea el movimiento del jugador manteniendo el giro de la cámara (vía **CinematicFallManager**), ilumina el número definido de baldosas con el **hintColor**, realiza un efecto de parpadeo intercalando los colores y luego restaura el estado visual original de la baldosa según el progreso real.

4.5.2 PathTile.cs

Componente fundamental del puzzle del Acto 1 que representa una única baldosa del suelo. Controla su propio estado visual (mediante iluminación emisiva dinámica) y físico (habilitando o deshabilitando su colisionador para dejar caer al jugador). Actúa como un sensor que notifica al `PathPuzzleManager` de los movimientos sobre la rejilla.

Parámetros de Inspector:

- **Colores de estado:** `robotColor` (morado, rastro de Jammo), `playerColor` (azul, paso correcto), `wrongColor` (rojo, paso erróneo), `defaultColor` (negro, apagada/invisible) y `hintColor` (cian, pista de memoria).
- **Renderizado:** `glowIntensity` (multiplicador de emisión de luz).
- **Físicas:** `solidCollider`, referencia al colisionador sólido (no-*trigger*) sobre el que camina el jugador.

Propiedades:

- `IsRobotLit (bool)`: indica si la baldosa está actualmente iluminada con el color del rastro del robot.

Métodos principales y Lógica:

- `Start()`: recupera el material de la baldosa y fuerza la habilitación de la palabra clave del *shader* `_EMISSION` (vital para que la luz funcione en la *build* final de Unity si no estaba marcada en el Editor).
- `SetColor(Color baseColor)`: ajusta los canales `_BaseColor` y `_EmissionColor` del material. Implementa una barrera de **idempotencia** (`_hasColor` y `_currentBase`) para descartar actualizaciones repetitivas y costosas, especialmente cuando se invoca desde el bucle de físicas.
- `SetSolid(bool)`: enciende o apaga el suelo físico. Cuando se desactiva, el jugador atraviesa la baldosa y cae.
- **Callbacks de Físicas (`OnTriggerEnter / Stay / Exit`):**
 - Distinguen las colisiones comprobando el *Tag* ("Robot" o "Player").
 - Durante la exploración libre (puzzle inactivo), iluminan temporalmente la baldosa y la apagan al salir.

- Durante el puzzle activo, delegan toda la lógica en el director llamando a `manager.CheckPlayerStep(this)` al entrar, e ignoran las salidas para evitar apagados accidentales. Bloquean cualquier cambio si el jugador está cayendo (`isPlayerFalling`).
- `ClearIfRobotLit()`: apaga la baldosa únicamente si su estado visual actual correspondía al de Jammo. Utilizado de forma masiva por el director para limpiar el rastro tras la animación de parpadeo.

4.5.3 DoorPortal.cs

Representa el portal físico de salida al final del Acto 1. Una vez que el puzzle se ha completado, este componente se encarga de bloquear el control del jugador y ejecutar una transición visual dramática (un efecto de “vórtice” o agujero negro) manipulando los efectos de post-procesado de la *Universal Render Pipeline* (URP) antes de cargar la siguiente escena.

Parámetros de Inspector:

- **Referencias:** `puzzleManager` (garantiza que el portal solo funcione si la propiedad `IsPuzzleCompleted` es verdadera) y `cinematicManager` (utilizado para aplicar un bloqueo **total** sobre el jugador: movimiento, cámara y combate).
- **Destino:** `sceneName (string)`, el nombre exacto de la escena a cargar (ej. "Act_02").
- **Efectos visuales:** `globalVolume (Volume)`, referencia al volumen global de URP de la escena del que se extraerán los *overrides*, y `transitionDuration` (duración del efecto en segundos).

Métodos principales y Lógica:

- `OnTriggerEnter(Collider)`: detecta la colisión física con el jugador. Si el puzzle está completado y la transición no se ha disparado aún, llama a `TriggerTransition()`.
- `TriggerTransition()`: punto de entrada principal (que también puede ser llamado externamente por el manager si la puerta se genera justo sobre la posición actual del jugador). Levanta un *flag* para evitar ejecuciones dobles, bloquea al jugador por completo y arranca la corrutina.
- `VortexTransitionRoutine()`: corrutina que conduce el efecto visual *frame* a *frame*:

1. Extrae los componentes `LensDistortion` y `ColorAdjustments` del `globalVolume`.
2. Utiliza una curva de interpolación cúbica (t^3) para que el efecto empiece de forma muy sutil y termine de manera violenta.
3. Gira progresivamente la cámara principal sobre su eje Z.
4. Modifica los valores de URP en tiempo real: lleva la intensidad de distorsión a -1 y reduce la escala visual a 0.01 (creando un efecto de “succión” hacia el centro de la pantalla), al mismo tiempo que baja la saturación a -100 para desaturar la imagen a blanco y negro.
5. Una vez concluido el bucle temporal, fuerza la carga asíncrona de la nueva escena mediante `SceneManager.LoadScene()`.

4.5.4 DoorBuildController.cs

Componente responsable de animar la “construcción” o revelación progresiva de la puerta del Acto 1. A medida que el jugador acierta los pasos del puzzle, este script hace emerger la puerta de abajo hacia arriba comunicándose con un *shader* personalizado. Destaca por su eficiencia al utilizar un `MaterialPropertyBlock`, evitando así la instanciación innecesaria de copias del material en memoria.

Parámetros de Inspector:

- **Referencias:** `doorRenderer` (`Renderer`), la malla de la puerta (ej. `SM_Arc`). Si se deja en blanco, el script intenta auto-asignarlo buscando en sus objetos hijos durante el `Awake`.
- **Animación (*Reveal*):** `tweenDuration` (`float`) define el tiempo en segundos que tarda la interpolación entre un paso y el siguiente; `verticalPadding` (`float`) es un margen de seguridad que se suma y resta a los límites (*bounds*) físicos de la malla para garantizar que en el paso 0 la puerta esté 100% oculta y en el paso final, 100% revelada.

Estado Interno:

- Almacena los límites calculados `_baseY` y `_topY`.
- `_propBlock`: instancia de `MaterialPropertyBlock` para inyectar datos al *shader* de forma limpia.

Métodos principales y Lógica:

- `Awake()`: Fase de inicialización. Obtiene el `Renderer`, lee sus dimensiones globales (`bounds.min.y` y `bounds.max.y`), aplica el *padding* vertical y configura la altura inicial de la puerta al mínimo.

- **SetProgressStep(int step, int totalSteps):** Método público (API) invocado por `PathPuzzleManager` cada vez que el jugador alcanza un hito. Calcula la altura objetivo mapeando la fracción del progreso (`step / totalSteps`) entre `_baseY` y `_topY`. Detiene cualquier animación en curso y arranca una nueva corrutina.
- **TweenBuildHeight(float target):** Corrutina privada que realiza la interpolación (*tweening*). Utiliza `Mathf.SmoothStep` para crear una curva de aceleración y desaceleración suave durante el tiempo definido por `tweenDuration`.
- **ApplyHeight():** Método auxiliar que extrae el bloque de propiedades actual del `Renderer`, inyecta el valor de la altura en la propiedad del *shader* `_BuildHeight` (cacheada previamente con `Shader.PropertyToID` para máximo rendimiento), y lo vuelve a aplicar a la malla.

4.5.5 JammoGuideController.cs

Componente que controla a Jammo en su papel de **guía automatizado** durante el Acto 1. Recorre físicamente el camino correcto generado procedimentalmente en dos fases (Inicio $\rightarrow$ *Checkpoint*, y *Checkpoint* $\rightarrow$ Fin), iluminando las baldosas a su paso para enseñar la ruta al jugador mientras este se encuentra bloqueado.

Parámetros de Inspector:

- **Configuración base:** `manager` (referencia al `PathPuzzleManager` para leer el arreglo de baldosas), `walkSpeed` (velocidad de traslación) y `jammoAnimator`.
- **Control de cinemática:** `cinematicManager`, utilizado para **bloquear** el movimiento del jugador (pero permitiendo el movimiento de la cámara/ratón) mientras Jammo camina.
- **Físicas y Pose:** `jammoBodyCollider` (colisionador sólido que se desactiva durante la caminata para que el jugador no pueda empujarlo ni desviar su trayectoria), `doorLookTarget` (punto de interés al que mirará al finalizar) y `turnSpeed` (velocidad de rotación).

Propiedades:

- **IsWalking (bool):** propiedad pública de solo lectura que indica si la corrutina de movimiento está activa. El director del puzzle la consulta para impedir que el jugador gaste “pistas de memoria” mientras Jammo ya está mostrando el camino.

Métodos principales y Lógica:

- `WalkToCheckpoint()` y `ResumeWalkToEnd()`: métodos públicos que inician el movimiento. Pasan los índices exactos del tramo a recorrer (del 0 a la mitad, o de la mitad al final) a la corrutina principal.
- `FollowPathRoutine(int startIndex, int endIndex)`: corrutina central que ejecuta la secuencia paso a paso:
 1. Bloquea al jugador (`SetPlayerMovement(false)`) y desactiva las colisiones físicas de Jammo.
 2. Inyecta el parámetro de velocidad en el Animator.
 3. Itera sobre el rango de baldosas correctas. Para cada una, interpola la rotación con `Quaternion.Slerp` hacia la dirección objetivo, y traslada la posición usando `Vector3.MoveTowards` ignorando el eje Y.
 4. **Coloración explícita:** Al no haber colisiones activas, la rutina no depende del evento `OnTriggerEnter` de las baldosas, sino que invoca directamente `SetColor(robotColor)` en cada baldosa alcanzada.
 5. Al llegar al final del último tramo, invoca a `TurnToFace` para mirar hacia la puerta.
 6. **Sincronización:** Cede el control a la corrutina `BlinkAndClearRobotTrail` del manager y espera a que termine de parpadear y borrar el rastro antes de devolverle el control de movimiento al jugador.
- `TurnToFace(Transform target)`: corrutina auxiliar que gira suavemente a Jammo solo en el plano XZ. Incluye un límite de seguridad (*timeout*) de 3 segundos para evitar bloqueos infinitos si el objetivo está exactamente en la misma coordenada geométrica.

4.5.6 PathGeneration/SelfAvoidingPathGenerator.cs

Generador procedimental de caminos **4-direccionales auto-evitantes de longitud exacta**. Emplea un algoritmo de búsqueda por retroceso (*backtracking*) altamente optimizado y aleatorizado. Se apoya en una severa poda matemática (distancia de Manhattan y paridad) y heurísticas predictivas para garantizar la generación de caminos válidos en unos pocos milisegundos sin cruces ni aglomeraciones.

Constantes de optimización:

- **NODE_BUDGET** (300 000): límite máximo de nodos expandidos por intento. Evita que el algoritmo se quede atrapado en ramas patológicas muy profundas, forzando un reinicio limpio.
- **MAX_ATTEMPTS_PER_CANDIDATE** (150): intentos máximos para generar un solo candidato antes de considerar que la ruta es inviable con la semilla actual.

Métodos principales y Lógica (El Motor de Búsqueda):

- **Generate(...)**: Punto de entrada público. Realiza una estricta validación previa de las reglas geométricas (comprueba si la distancia de Manhattan y la paridad son compatibles con los pasos totales solicitados). Genera múltiples candidatos, los evalúa con **FuzzyPathEvaluator** y devuelve el que obtenga la mejor puntuación estética.
- **SearchOne(Vector2Int)**: Lanza hasta **MAX_ATTEMPTS_PER_CANDIDATE** intentos de búsqueda en profundidad (**Recurse**), reiniciando el presupuesto de nodos y las listas de celdas visitadas en cada iteración.
- **Recurse(Vector2Int current)**: Corazón del *backtracking*. Para evitar la asignación de memoria en cada llamada (*Zero Allocs*), utiliza búferes de direcciones preasignados por nivel de profundidad (**_dirBuffers**). Antes de expandir nodos, aplica la **Heurística de Warnsdorff**: mezcla aleatoriamente las direcciones y luego las ordena priorizando las celdas vecinas con el *menor* grado de libertad, logrando una rápida convergencia hacia caminos válidos.

Reglas de Poda (*Pruning*) en **Recurse**:

1. **Cierre inalcanzable**: Si la distancia de Manhattan al objetivo supera los movimientos restantes, o sus paridades difieren, la rama se descarta instantáneamente.
2. **Objetivo prematuro**: La celda final solo puede ser tocada en el último paso exacto.
3. **Separación (Vecindad 8)**: Usando **IsSpaced(...)**, asegura que los brazos paralelos del camino no se toquen ni siquiera en diagonal, garantizando un recorrido “limpio” visualmente.
4. **Condición de Penúltima Baldosa**: Obliga a que una celda específica (**_mandatedPenult**) ocurra exclusivamente en el índice $(N - 2)$, fijando así la dirección de entrada a la puerta final.
5. **Anti-escaleras**: Compara los vectores de dirección históricos para prohibir dos giros consecutivos, forzando rectas mínimas entre curvas.

- **FreeDegree(Vector2Int)**: Cuenta cuántas celdas ortogonales vecinas están libres, alimentando la heurística de Warnsdorff.

4.5.7 PathGeneration/FuzzyPathEvaluator.cs

Implementa un motor de inferencia difusa (estilo Mamdani simplificado y dedicado) diseñado para evaluar la **calidad estética** de los caminos generados por el algoritmo de *backtracking*. Actúa como la *función de aptitud* (fitness) que permite generar múltiples candidatos válidos y seleccionar el que ofrezca la mejor experiencia de juego, premiando caminos amplios y equilibrados, y penalizando los que son demasiado rectos, erráticos o que se pegan demasiado a los bordes de la cuadrícula.

Entradas Nítidas (Crisp Inputs) y Fuzzificación: El evaluador extrae tres métricas geométricas del camino y las mapea a conjuntos difusos mediante funciones de pertenencia triangulares ($f(x, a, b, c)$):

- **Densidad de giros (turnDensity)**: Calcula la proporción de giros respecto a los movimientos totales ($\frac{\text{turns}}{\text{moves}-1}$). Se fuzzifica en tres conjuntos: *Straight* (Recto), *Balanced* (Equilibrado) y *Wiggly* (Errático/Zig-zag).
- **Proximidad a los bordes (edgeProximity)**: Calcula la distancia media de Chebyshev desde el camino hacia los límites del tablero, normalizada en $[0, 1]$. Se divide en los conjuntos: *Far* (Lejos) y *Near* (Cerca).
- **Dispersión (spread)**: Compara el área de la caja delimitadora (*bounding box*) que ocupa el camino contra la cantidad de celdas del mismo. Una proporción cercana a 1 implica que el camino está apelotonado; una mayor, que aprovecha bien el espacio. Se divide en: *Cramped* (Apretado) y *Spacious* (Espacioso).

Base de Reglas (Mamdani): El evaluador aplica 7 reglas lingüísticas combinando los antecedentes con el operador **min** lógico (AND). Las activaciones resultantes se agrupan mediante el operador **max** para determinar la fuerza de los tres conjuntos de salida: *LOW*, *MEDIUM* y *HIGH*.

1. **SI** es *Balanced* **Y** *Spacious* $\rightarrow$ **ENTONCES** calidad *HIGH*.
2. **SI** es *Straight* $\rightarrow$ **ENTONCES** calidad *LOW* (aburrido).
3. **SI** es *Wiggly* **Y** *Near* $\rightarrow$ **ENTONCES** calidad *LOW* (caótico y pegado al borde).
4. **SI** es *Far* **Y** *Spacious* $\rightarrow$ **ENTONCES** calidad *HIGH* (centrado y amplio).

5. **SI** es *Balanced* **Y** *Far* $\rightarrow$ **ENTONCES** calidad *HIGH*.
6. **SI** es *Cramped* $\rightarrow$ **ENTONCES** calidad *LOW* (apelotonado).
7. **SI** es *Wiggly* **Y** *Spacious* $\rightarrow$ **ENTONCES** calidad *MEDIUM* (salva los caminos erráticos si al menos usan bien el espacio).

Defuzzificación: Al igual que la IA de los enemigos, utiliza el método del **centroide**. Para mayor eficiencia, al ser un evaluador dedicado, no emplea el *Builder* genérico, sino un bucle estático optimizado a 20 pasos de integración (*DefuzzSteps*) sobre el dominio de salida $[0, 1]$. Devuelve un valor nítido (*crisp score*) final que el *SelfAvoidingPathGenerator* utiliza para quedarse con el mejor candidato.

Fragmento de la Base de Reglas Estáticas:

```

1 // Rule base. Activation = min(antecedents). Consequent = linguistic label of the output set.
2 float r1 = Mathf.Min(balanced, spacious); // HIGH
3 float r2 = straight; // LOW
4 float r3 = Mathf.Min(wiggly, near); // LOW
5 float r4 = Mathf.Min(far, spacious); // HIGH
6 float r5 = Mathf.Min(balanced, far); // HIGH
7 float r6 = cramped; // LOW
8 float r7 = Mathf.Min(wiggly, spacious); // MEDIUM
9
10 float highActivation = Mathf.Max(r1, Mathf.Max(r4, r5));
11 float mediumActivation = r7;
12 float lowActivation = Mathf.Max(r2, Mathf.Max(r3, r6));
13
14 return Defuzzify(lowActivation, mediumActivation, highActivation);

```

Listing 2: Evaluación de reglas lingüísticas en *FuzzyPathEvaluator*

4.5.8 PathGeneration/GridPath.cs

Secuencia ordenada de celdas (*List<Vector2Int> Cells*). API: Add, Contains, Concat, First, Last, Count.

4.6 Carpeta: Level_2 (Acto 2 — Arena y Estatua)

4.6.1 WaveManager.cs

Motor central de **oleadas** del Acto 2. Diseñado para ser dirigido externamente: el director de ensamblaje (*StatueAssemblyDirector*) y el *Act02Director* inician y detienen el ciclo. Gestiona el ciclo de vida de los enemigos de forma eficiente mediante la técnica de *Object Pooling*, coordina los puntos de aparición y administra un sistema de *drops* (botines) de curación con probabilidades compuestas.

Parámetros de Inspector (Selección):

- **Configuración de Oleadas (Wave):** Array de objetos que definen cada oleada. Incluyen el muro de aparición (`spawnWall`), el prefabricado del enemigo, la cantidad y parámetros de distribución espacial (`spawnSpread`, `spawnDepth`) y temporal (`spawnInterval`).
- **Distribución y *Pooling*:** `minSpawnSeparation` y `spawnPlacementTries` implementan un *rejection sampling* que evita que los enemigos aparezcan solapados en el mismo *frame*. `corpseDelay` dicta el tiempo de espera antes de devolver el enemigo muerto al *pool*.
- **Botines de Salud (*Drops*):** `healthDropChance` es la probabilidad absoluta de que un enemigo suelte un botín al morir. Si se cumple, `jammoDropShare` decide la probabilidad de que ese botín sea para Jammo en lugar de para el jugador, utilizando prefabs distintos. `healthDropHeightY` fuerza una altura de aparición constante para evitar que las animaciones de muerte entierren el objeto.
- **Bucle y Seguridad:** `loopWaves` (para repetir la lista hasta que se monte la estatua), `delayBetweenWaves` y `maxWaveDuration` (un mecanismo de seguridad anti-*softlock* que aborta la oleada si no se limpia en un tiempo prudencial, por ejemplo si un enemigo cae al vacío o queda inalcanzable).

Eventos (C# y Unity):

- **EnemyKilled (Acción C#):** El gancho crucial al que se suscribe el `StatueAssemblyDirector` para encolar nuevas piezas de la estatua por cada enemigo abatido.
- **WaveCompleted y HealthDropSpawned:** Eventos para el sistema de puntuación.

Métodos principales y Lógica:

- **StartNextWave():** Avanza el índice (con vuelta a 0 si `loopWaves` es verdadero) y arranca la corrutina principal de generación.
- **SpawnWaveRoutine(Wave, int):** Ejecuta la oleada. Si no existe, crea un `SimplePool` para el prefabricado. Para cada enemigo: calcula una posición válida con `PickSpawnPosition`, recupera una instancia del *pool*, inyecta una suscripción al evento de muerte (`health.Died += OnEnemyDied`) garantizando que no se duplique (`_wired`), y resetea su estado invocando `ai.Configure()`.

- **OnEnemyDied(GameObject):** Ejecuta el *drop* de salud mediante una doble tirada (probabilidad absoluta → probabilidad relativa). Dispara **EnemyKilled** e inicia la cuenta regresiva para liberar el cadáver de vuelta al *pool* (**ReleaseAfter**). Si el recuento de vivos llega a 0, declara la oleada superada (**WaveCleared()**).
- **WaveCleared():** Dispara los eventos de fin de oleada, cancela la corrutina de *timeout* y gestiona el auto-avance si está configurado.

4.6.2 StatueAssemblyDirector.cs

Director de ensamblaje del Acto 2. Funciona como puente **desacoplado** entre el generador de oleadas (**WaveManager**) y la inteligencia artificial del robot acompañante (**JammoCarrier**). Recibe las notificaciones de los enemigos caídos y las encola; de forma asíncrona, orquesta los paseos y animaciones de recolección de Jammo hasta completar la estatua.

Parámetros de Inspector:

- **Referencias fundamentales:** **waveManager** (para suscribirse a las muertes), **statueRig** (el armazón fantasma a rellenar), **pieceSpawner** (generador de las piezas físicas transportables) y **jammo** (el controlador físico del compañero).
- **Geometría de entrega:** **placementPoint** define la coordenada exacta frente al pedestal a la que debe caminar Jammo antes de entregar la pieza.
- **Tiempos y Eventos:** **pauseAfterPlace** (tiempo de respiración de Jammo tras dejar una pieza) y los **UnityEvent** (**onAssemblyStarted** y **onAssemblyFinished**) para disparar las cinemáticas o notificar al **Act02Director**.

Estado Interno (La Cola Asíncrona):

- Utiliza una variable entera **_pending** para registrar las muertes de los enemigos. Este contador nunca supera la cantidad de huecos (*slots*) restantes en la estatua, de modo que las muertes en exceso no desbordan la lógica.

Métodos principales y Lógica:

- **OnEnemyKilled():** Método asociado (*hook*) al delegado C# de **WaveManager**. Su única responsabilidad es sumar +1 al contador **_pending**.
- **RunRoutine():** La máquina de estados principal (corrutina infinita) que dicta las acciones de Jammo. Si **_pending > 0**, descuenta uno y ejecuta la **secuencia de transporte**:

1. Interroga al `statueRig` para reservar un índice de *slot* vacío aleatorio.
2. Spawnea la pieza física correspondiente (`PieceSpawner`).
3. Cede el control a la corrutina de navegación de Jammo (`WalkTo`) hacia el objeto.
4. Arranca la corrutina de recogida (`PickUpRoutine`) y luego ordena a Jammo caminar hasta el `placementPoint`.
5. **Manejo de interrupciones:** Si Jammo muere en el trayecto, la rutina cancela la entrega, devuelve el hueco a la estatua para que vuelva a estar disponible y rompe el ciclo. Si Jammo recibe daño (`DropRequested`), la pieza se da por perdida, se devuelve el *slot* y Jammo vuelve a empezar.
6. Si llega a salvo, libera la pieza (`ReleasePiece`), notifica a la estatua para que materialice ese brazo/pierna y espera un momento (`pauseAfterPlace`).

Si la cola está vacía, ordena a Jammo volver a su posición de inicio y esperar.

- Al finalizar la estatua, invoca el evento de éxito y detiene las oleadas indefinidamente.

4.6.3 StatueRig.cs

Controla la representación visual y el estado lógico de la estatua gigante del Acto 2. En lugar de calcular complejas alineaciones espaciales para ensamblar piezas tridimensionales, la estatua es el propio modelo completo y *riggeado* de Jammo (escalado $\times 10$). El ensamblaje se simula cambiando los materiales de las distintas mallas ya posicionadas, pasando de un estado transparente a uno sólido.

Estructuras de datos y Parámetros de Inspector:

- **Slot (Clase anidada):** Define una parte ensamblable. Contiene una referencia al `Renderer` (`bigStatuePart`), un *flag* para saber si requiere el material especial de los ojos (`useEyesMaterial`), y un estado interno `filled`.
- **Materiales:** `ghostMaterial` (material base transparente/holográfico), `solidMaterial` (metal estándar de Jammo) y `solidMaterialEyes` (material emisivo específico para las pantallas faciales).
- **Eventos de Unity:** `onPartRevealed` y `onStatueComplete`, utilizados para el HUD y efectos visuales.

Propiedades y Eventos (C#):

- **Eventos:** `PartRevealed` y `StatueCompleted` (delegados de C# que replican los `UnityEvents` para suscripciones de código limpio, vitales para el `ScoreManager`).
- **Estado:** `RemainingCount`, `IsComplete`, `TotalSlots`, `FilledCount` y `Normalized`. Este último devuelve el progreso exacto en $[0, 1]$ para alimentar la barra de progreso (`StatueProgressBar`).

Métodos principales y Lógica:

- `Awake()`: Fase de inicialización. Recorre todos los `slots`, les asigna forzosamente el `ghostMaterial` inicial y llena una lista interna (`_free`) con los índices disponibles.
- `TakeRandomUnfilledSlot()`: Reserva una pieza para que Jammo vaya a buscarla. Utiliza un algoritmo de eliminación de tiempo constante $O(1)$ (*swap-last*): escoge un índice aleatorio de la lista `_free`, lo intercambia con el último elemento de la lista y luego elimina el último.
- `PartNameOf(int index)`: Devuelve el nombre exacto del `GameObject` de ese *slot*. Este nombre actúa como **clave de correlación** para que el `PieceSpawner` sepa qué parte debe hacer visible en el modelo en miniatura que transporta Jammo.
- `OnSlotFilled(int index)`: Se llama cuando Jammo entrega la pieza con éxito. Realiza el cambio de `sharedMaterial` (verificando si usa el material estándar o el de los ojos) y dispara los eventos de progreso/victoria.
- `ReturnSlot(int index)`: Se invoca si Jammo es golpeado o muere mientras transporta la pieza. Como `OnSlotFilled` nunca llegó a ejecutarse, la malla grande sigue en estado fantasma; este método simplemente devuelve el índice a la lista `_free` para que el director pueda volver a sortearla en el futuro.

4.6.4 PieceSpawner.cs

Genera las “piezas” transportadas (cada una es una instancia completa del Jammo escala 1 con `JammoPartSet`, mostrando solo una parte). Pooled. **Parámetros:** `scale1JammoPrefab`, `spawnPoints`, `poolSize`.

Métodos: `SpawnPiece(string partName)`, `Release(GameObject)`.

4.6.5 PickupRotator.cs

Hace girar un pickup en Y y opcionalmente lo hace flotar (seno). **Parámetros:** `rotationSpeed`, `enableBobbing`, `bobbingSpeed`, `bobbingAmount`. **Método:** `Update()`.

4.6.6 JammoCarrier.cs

Controlador principal de la Inteligencia Artificial de Jammo durante la arena de combate del Acto 2. Combina un sistema de locomoción híbrido (basado en `NavMeshAgent` para la evasión de obstáculos con un *fallback* a movimiento cinemático) y gestiona los estados visuales del transporte de piezas. Implementa la interfaz `IDamageReaction` para introducir una mecánica de vulnerabilidad: si Jammo recibe daño mientras transporta una pieza, la deja caer y el esfuerzo se pierde.

Parámetros de Inspector:

- **Locomoción:** `runSpeed`, `turnSpeed`, `turnInPlaceThreshold` (ángulo mínimo para forzar una rotación en el sitio antes de avanzar, evitando deslizamientos irreales) y `arriveDistance`.
- **Animación:** `locomotionSpeedParam` enlaza la velocidad real en m/s con el *blend tree* del `Animator`, asegurando que el ciclo de caminado coincida exactamente con el desplazamiento físico.
- **Transporte (*Carrying*):** `carryAnchor` (punto de anclaje de la pieza levitante), `bobAmplitude` y `bobFrequency` (parámetros de la oscilación senoidal), `liftDuration` y `placeDuration` (tiempos de interpolación al recoger y soltar).
- **Estado:** `homePost` (posición de descanso a la que vuelve cuando no hay piezas pendientes) y `activateOnStart`.

Propiedades de Estado (Lectura Pública):

- `IsCarrying` (bool): Indica si actualmente tiene una pieza anclada.
- `IsDead` (bool): Detiene la evaluación de las corrutinas si la salud llega a 0.
- `DropRequested` (bool): *Flag* que se activa exclusivamente desde `OnDamaged` si Jammo es golpeado mientras `IsCarrying` es verdadero. El director lee este flag para abortar el ciclo lógico de entrega.

Métodos principales y Lógica:

- `WalkTo(Vector3)`: Corrutina de navegación. Intenta utilizar el `NavMeshAgent`. Para evitar que el movimiento parezca mecánico, Jammo primero rota en el sitio hacia su objetivo y luego avanza. Incluye un temporizador **anti-atascos** (*anti-stall*): si la ruta está bloqueada pero Jammo no logra avanzar durante 2 segundos continuos, la corrutina se rompe para evitar un *softlock* en el director.

- **PickUpRoutine(Transform):** Levanta la pieza desde el suelo hasta el `carryAnchor` utilizando una curva de suavizado cúbico (*ease-out*). Una vez en posición, arranca la corrutina `BobRoutine` para aplicar un efecto de levitación matemática (`Mathf.Sin`).
- **Rutinas de Resolución de Piezas:** Maneja tres escenarios distintos de forma modular:
 1. **ReleasePiece():** Entrega exitosa a la estatua gigante. Desvincula la pieza de Jammo y la encoge a escala 0 (desaparece) porque la estatua visualiza su propia malla mediante el `StatueRig`.
 2. **PlacePiece():** Entrega en la que la pieza **conserva** su escala original (ej. apilado de piezas sobre un altar). Se desplaza hacia el objetivo y se desvincula.
 3. **DropCarriedPiece():** Pieza perdida por daño. La pieza cae al suelo (`Vector3.down`) y se desvanece progresivamente, informando visualmente al jugador del fracaso.

Fragmento de la lógica de locomoción y Anti-Atascos:

```

1  // wait for the path computation, then for arrival. Anti-stall timer:
2  // if the path is blocked/partial and Jammo isn't advancing, continue
3  // anyway (no deadlock of the director's coroutine).
4  float stuck = 0f;
5  while (_agent.pathPending || _agent.remainingDistance > _agent.stoppingDistance)
6  {
7      if (IsDead || _dropRequested) break; // death or lost piece
8
9      SetLocomotion(_agent.velocity.magnitude); // Sync Animator
10
11     stuck = _agent.velocity.sqrMagnitude > 0.01f ? 0f : stuck + Time.deltaTime;
12     if (stuck > 2f)
13     {
14         Debug.LogWarning("[JammoCarrier]_Path_blocked:_continuing.", this);
15         break;
16     }
17     yield return null;
18 }

```

Listing 3: Extracto de WalkTo en JammoCarrier

4.6.7 JammoHealth.cs

Componente análogo a `PlayerHealth`, diseñado específicamente para gestionar la secuencia de muerte del compañero robot (Jammo) en la arena del Acto 2. Dado que la supervivencia de Jammo es una condición de victoria, este script coordina la

transición hacia el estado de derrota (*Game Over*), asegurando que las animaciones se reproduzcan antes de detener la lógica del juego.

Parámetros de Inspector:

- **onJammoDied** (UnityEvent): Evento crucial invocado **después** de la animación de muerte. Desde el Inspector se enlaza a la interfaz de usuario (`MainMenuManager.ShowGameOver`) y al director de la arena (`WaveManager.StopAndEnd`) para abortar las oleadas de enemigos.
- **Animación:** `jammoAnimator` (referencia al `Animator`, con auto-búsqueda en el `Awake` si se deja vacío), `deathTriggerName` (nombre del parámetro, por defecto "Die") y `deathAnimationDuration` (segundos de margen para mostrar la caída antes de oscurecer la pantalla).

Propiedades:

- **Health:** Propiedad pública que expone el componente `Health` subyacente para facilitar el acceso a otros scripts vinculados.

Métodos principales y Lógica:

- **HandleDied():** Punto de entrada público enlazado en el Inspector al evento `Health.onDied`. Implementa una barrera de seguridad mediante el *flag* `_isDying` para garantizar que la secuencia de muerte arranque una única vez, evitando solapamientos si Jammo recibe daño masivo en un solo *frame*.
- **DeathRoutine():** Corrutina privada que ejecuta la secuencia:
 1. Valida dinámicamente la existencia del parámetro de animación utilizando el `AnimatorParameterCache` (evitando advertencias en consola si el *Animator* cambia) y dispara el *trigger*.
 2. Espera la cantidad de segundos definida por `deathAnimationDuration`. Utiliza `WaitForSecondsRealtime` de forma deliberada para asegurar que el tiempo transcurra independientemente de si otro sistema ha alterado el `Time.timeScale`.
 3. Finalmente, invoca `onJammoDied.Invoke()` para notificar al resto de directores que el nivel ha terminado en fracaso.

4.6.8 JammoPartSet.cs

Va en el prefab Jammo escala 1 usado como “pieza”: `ShowOnly(string partName)` muestra una sola parte, oculta las demás y **congela la pose** (desactiva el `Animator`).

Parámetros: `parts` (renderers), `animator`.

4.6.9 Act02Director.cs

Director principal del Acto 2. Se encarga de orquestar la secuencia lineal de la escena, integrando la narrativa con los sistemas de combate. Divide el flujo en tres grandes fases: un **Prólogo** cinematográfico, la fase de **Gameplay** (delegada al **WaveManager**) y un **Epílogo** que concluye el acto y realiza la transición al nivel final.

Parámetros de Inspector:

- **Cámaras (*Cinemachine*):** `prologueCamera` y `gameplayCamera`. Utilizan la clase base abstracta `CinemachineVirtualCameraBase` para garantizar la compatibilidad tanto con `Cinemachine 2` como con la versión 3. `activePriority` e `idlePriority` definen los pesos para las transiciones.
- **Narrativa y Flujo:** `prologueDialogue` y `epilogueDialogue` (referencias a `DialogueAsset`), y `nextSceneName` (cadena con el nombre exacto de la escena a cargar, ej. "Act_03").
- **Tiempos (*Timing*):** Variables críticas para la fluidez cinematográfica: `initialPauseOnPlayer`, `prologueBlendDuration`, `pauseBeforeWavesStart` y `pauseBeforeNextScene`.
- **Interfaz (HUD):** `hudReveals` es un arreglo de componentes de UI (como la barra de progreso de la estatua) que se ocultan durante las cinemáticas y se revelan justo antes del combate.

Métodos principales y Lógica (Flujo de la Escena):

- `RunPrologue()`: Corrutina que arranca automáticamente en el `Start()`. Primero, bloquea el *input* del jugador mediante el `DialogueManager`. Luego, altera temporalmente la interpolación predeterminada (*DefaultBlend*) del `CinemachineBrain` para lograr un paneo lento y dramático hacia la estatua. Una vez allí, reproduce el diálogo inicial. Al terminar, la cámara vuelve al jugador, el HUD se revela con una animación de entrada y se instruye al `WaveManager` para que comience el combate.
- `OnAssemblyFinished()`: Método público (enlazado desde el Inspector al `UnityEvent` del `StatueAssemblyDirector`). Protegido por un *flag* para garantizar una única ejecución, sirve como puente entre el fin del `gameplay` y el inicio del cierre narrativo.
- `RunEpilogue()`: Corrutina simétrica al prólogo. Oculta el HUD, devuelve el enfoque a la estatua gigante (ahora completa), reproduce el diálogo final

y, tras una pausa de absorción, restablece las medidas de seguridad globales (`Time.timeScale = 1f` y la liberación del cursor) antes de ordenar la carga de la siguiente escena.

Fragmento Técnico: Sobrescritura dinámica de *Cinemachine*:

```
1 // Slow camera blend -> statue. Override the Brain's DefaultBlend for the
2 // requested duration, then restore it safely.
3 CinemachineBlendDefinition prevBlend = default;
4 bool blendOverridden = false;
5 if (_brain != null && prologueBlendDuration > 0f)
6 {
7     prevBlend = _brain.DefaultBlend;
8     _brain.DefaultBlend = new CinemachineBlendDefinition(prevBlend.Style, prologueBlendDuration)
9     ;
10    blendOverridden = true;
11 }
12 SetActiveCamera(prologueCamera);
13 yield return WaitForBlend();
14
15 if (blendOverridden && _brain != null) _brain.DefaultBlend = prevBlend;
```

Listing 4: Inyección de un blend temporal en `Act02Director`

4.6.10 `VortexEnterTransition.cs`

Reproduce un **vórtice inverso** al entrar en `Act_02` (parte de los valores extremos de distorsión/desaturación y los suaviza hasta lo normal). **Parámetros:** `globalVolume`, `transitionDuration`. **Método:** `ReverseVortexRoutine()`.

4.7 Carpeta: `Level_3` (Acto 3 — El Espejo de Agua)

4.7.1 `MirrorDuelDirector.cs`

El director supremo del Acto 3 (“El Espejo de Agua”). Orquesta el duelo final estructurando el combate en dos fases temáticas y secuenciales (Sol y Luna). Este componente centraliza el control sobre los actores (Jugador, Jefe y Jammo), manipula el sistema de cámaras (*Cinemachine*), gestiona los diálogos narrativos e inyecta reglas de combate asimétricas en tiempo real.

Parámetros de Inspector (Selección):

- **Actores y Reglas:** Referencias a los componentes de salud e IA (`boss`, `bossShield`, `jammo`). `phase1DamageFloor` establece el límite de daño de la primera fase (ej. 0.5 para detener la vida del jefe al 50%), y `phase1PieceCount` dicta cuántas piezas se exigen antes de habilitar el daño.

- **Gestión de Piezas:** `enemyPieces` y `playerAltarSlots`. A diferencia del Acto 2, aquí las piezas son *props* persistentes en la escena (no hay *pooling*), optimizando el rendimiento. El director simplemente altera su jerarquía y posición.
- **Cinematografía y Cámaras:** `overviewCamera` y `gameplayCamera`. Utilizan `CinemachineVirtualCameraBase` para mantener compatibilidad entre versiones de Cinemachine. Los tiempos de interpolación (`cinematicBlendDuration`) y los puntos de anclaje (`playerSpawn`, `celestialLookTarget`) definen la coreografía de las transiciones.
- **Efectos Visuales:** `dissolveDuration` y `maxDissolveRadius` controlan el colapso del mundo al ganar el juego, comunicándose directamente con el *shader* global.

Flujo del Combate (La Máquina de Estados): El duelo (`RunDuel`) ejecuta dos veces la corrutina `PhaseRoutine`, alternando las mecánicas:

1. **Fase de Recolección:** El jefe es resistente al daño (tope de 1 HP por golpe). Cada cierto número de aciertos, su escudo cede (`NotifyPlayerBrokePiece()`) y desbloquea una pieza. Jammo viaja al altar enemigo para recogerla y llevarla al altar del jugador.
2. **Ventana de Daño (*Damage Window*):** Al entregar todas las piezas de la fase, el jefe se vuelve vulnerable y el jugador puede reducir su vida hasta el tope de fase.
3. **Diferencia Sol vs. Luna:** En la fase Sol, Jammo es ignorado por el jefe. En la fase Luna, el jefe se potencia, la IA intercepta a Jammo, y si este recibe daño mientras transporta una pieza, el progreso se pierde y la pieza regresa al altar enemigo (`ReturnPieceHome`).

Métodos principales y Lógica Cinematográfica:

- `RunPhaseTransition()`: Corrutina que marca el ecuador del combate. Bloquea al jugador (desactivando directamente el `CharacterController` y el *Input* para mayor seguridad), eleva la cámara a vista de pájaro (`overviewCamera`), obliga a los tres actores a caminar hacia sus marcas iniciales (`WalkActorsToSpawn`) y los hace girar para contemplar el cielo (`TurnActorsToFace`) antes de invocar el giro del mundo (`MirrorFlipDirector.FlipTo`).

- `CarryOnePiece(...)`: Mueve a Jammo entre los altares. Si la fase es la Luna y `jammo.DropRequested` es verdadero (recibió un golpe), ejecuta `DropCarriedPiece`, devuelve la pieza a su origen e invoca el evento `PieceLost` para penalizar la puntuación.
- `DissolveWorldAroundBoss()`: Ejecuta el efecto de disolución final. En lugar de iterar sobre los materiales de la escena, modifica propiedades globales del *shader* (`_GlobalPlayerPos` y `_GlobalDissolveRadius`), logrando un efecto de colapso esférico altamente optimizado desde el cadáver del jefe hacia afuera.

Fragmento Técnico: Bloqueo Robusto del Jugador:

```

1 // Deshabilita el Input y los scripts de movimiento/combate cacheados.
2 // Esto garantiza que el jugador quede congelado en el primer frame de la cinemtica,
3 // incluso si el DialogueManager no estuviera instanciado en esta escena.
4 if (_playerInput != null) _playerInput.enabled = !locked;
5 if (_playerLockScripts != null)
6 {
7     for (int i = 0; i < _playerLockScripts.Length; i++)
8     {
9         if (_playerLockScripts[i] != null) _playerLockScripts[i].enabled = !locked;
10    }
11 }
12 if (locked) SetPlayerLocomotion(0f); // Congela el blend tree del Animator

```

Listing 5: Inhabilitación de componentes para secuencias cinemáticas seguras

4.7.2 BossDuelAI.cs

Inteligencia Artificial del jefe final (*doppelgänger*). A diferencia de los enemigos de la arena, el jefe opera de forma independiente (sin `NavMesh` ni coordinador) empleando locomoción basada en `CharacterController`. Su diseño evita la monotonía del combate combinando una **Máquina de Estados Finitos (FSM) basada en ventanas temporales** (que dicta el ritmo estratégico) con una capa superior de **defensa reactiva** en tiempo real.

Parámetros de Inspector (Selección):

- **Movimiento y Locomoción:** `walkSpeed`, `runSpeed` y `runDistance`. La velocidad real alimenta dinámicamente un *blend tree* en el `Animator` para transicionar suavemente entre andar y correr sin que los pies se deslicen.
- **Ritmo de Combate (Ventanas):** `aggroWindow` (tiempo de ataque continuo), `guardWindow` (tiempo de espera en guardia) y `repositionRadius`. Estas variables obligan al jefe a alternar entre agresión, reposicionamiento táctico y defensa pasiva.

- **Defensa Reactiva:** `reactiveBlockChance` (probabilidad de bloqueo instintivo), `reactiveBlockTime` (duración del bloqueo) y `reactiveBlockRange` (distancia máxima para reaccionar).
- **Potenciación de Fase (Luna):** `moonSpeedMultiplier`, `moonAttackDamage` y umbrales difusos como `interceptThreshold` y `seekHealthThreshold`.

Máquina de Estados y Comportamientos:

1. **Aggro:** Persigue al objetivo y ejecuta una corrutina de combo estático (`ComboRoutine`) al entrar en rango.
2. **Reposition:** Corre hacia un punto aleatorio circundante (`PickRepositionPoint`) calculado con una distancia mínima de seguridad para forzar movimiento real.
3. **Guard:** Se detiene, levanta el escudo (`BossShield`) y mira al jugador esperando que se acerque (`engageDistance`).
4. **Kite:** Táctica de rodeo lateral manteniendo la distancia (`kiteDistance`) y cambiando la dirección orbital periódicamente.
5. **SeekHealth:** Prioridad máxima condicional. Navega rápidamente hacia un botón de salud activo (`DuelHealthSpawner`).

Métodos principales y Lógica Avanzada:

- `Update()`: Actúa como *dispatcher* (despachador). Primero evalúa la capa reactiva (`DetectReactiveBlock`). Si el jefe está en un combo o bloqueando, anula el movimiento. Luego evalúa la necesidad de curarse (difusa). Si nada de lo anterior toma prioridad, actualiza el estado activo de la FSM (ej. `TickAggro`).
- `DetectReactiveBlock()`: Lee el componente del jugador (`PlayerMeleeAttack.IsSwinging`) buscando un **flanco de subida** (*rising edge*). Si el jugador inicia un ataque de frente y en rango, el jefe lanza una tirada probabilística para levantar instantáneamente la guardia, creando una sensación de reflejos orgánicos.
- `ResolveTarget()`: Selecciona entre el Jugador o Jammo. En la Fase Luna, interroga al motor de inferencia (`BossFuzzyBrain`) para decidir si intercepta a Jammo. Implementa **histéresis**: una vez que decide perseguir a Jammo, mantiene la fijación hasta que la distancia supere `interceptGiveUpDistance`, evitando que el jefe oscile de objetivo en cada *frame*.

- **ReturnToSpawn(Vector3):** Corrutina de caminata cinemática empleada por el **MirrorDuelDirector** para reposicionar al jefe con gracia (y animaciones correctas) durante la transición de fase, ignorando la IA estándar mientras está pausado (**_paused**).

Fragmento Técnico: Intercepción Difusa con Histéresis:

```

1 float jammoDist = Planar(jammo.transform.position - transform.position).magnitude;
2 _decisionTimer -= Time.deltaTime;
3
4 if (_decisionTimer <= 0f)
5 {
6     _decisionTimer = decisionInterval;
7     // La logica difusa HABILITA la intercepcion; la histeresis (abajo) la apaga.
8     if (!_interceptJammo && _brain.Intercept(jammoDist, true) >= interceptThreshold)
9         _interceptJammo = true;
10 }
11
12 // Histeresis: una vez enganchado, persigue a Jammo mientras la distancia sea aceptable.
13 // Si Jammo se aleja demasiado, suelta el objetivo para evitar atascos.
14 if (_interceptJammo && jammoDist <= interceptGiveUpDistance)
15     return jammo.transform;
16
17 _interceptJammo = false;
18 return player;

```

Listing 6: Evaluación de objetivo con lógica difusa e histéresis

4.7.3 BossFuzzyBrain.cs

Implementa la lógica difusa **dedicada** para la toma de decisiones estratégicas del jefe final. A diferencia de los enemigos de la arena del Acto 2, el jefe utiliza su propio cerebro difuso aislado. Esta separación arquitectónica garantiza que el ajuste (*tuning*) de las curvas de pertenencia del jefe no altere el comportamiento de la horda, y viceversa.

El cerebro encapsula **dos controladores difusos independientes**, contruidos una sola vez y almacenados estáticamente (**_sharedIntercept** y **_sharedSeekHealth**). Para garantizar un rendimiento óptimo (*Zero Allocs*), la clase preasigna arreglos **float[]** para las entradas y salidas, y cachea los índices de las variables en el constructor.

1. Controlador de Intercepción (Intercept): Evalúa el deseo del jefe de interrumpir su combate contra el jugador para detener a Jammo.

- **Entradas (*Inputs*):**

- **jammoDistance [0,16]:** Distancia espacial hacia Jammo. Conjuntos: *Near* (Cerca) y *Far* (Lejos).

- jammoCarrying [0,1]: Entrada booleana tratada como continua. Conjuntos: *No* y *Yes*.
- **Salida (*Output*):** intercept [0,1], dividido en *Low* y *High*.
- **Base de Reglas:**
 1. **SI** Jammo está *Near* **Y** *Yes* (transporta pieza) → Deseo *High*.
 2. **SI** Jammo está *Far* → Deseo *Low* (no vale la pena abandonar al jugador).
 3. **SI** *No* (con las manos vacías) → Deseo *Low* (lo ignora).

2. Controlador de Curación (*SeekHealth*): Evalúa la urgencia de abandonar el combate para recoger un botín de salud neutral en la arena.

- **Entradas (*Inputs*):**
 - selfHealth [0,1]: Vida normalizada del jefe. Conjuntos: *Low* (Herido) y *High* (Sano).
 - pickupDistance [0,16]: Distancia al botín más cercano. Conjuntos: *Near* y *Far*.
- **Salida (*Output*):** seekHealth [0,1], dividido en *Low* y *High*.
- **Base de Reglas:**
 1. **SI** vida *Low* **Y** botín *Near* → Deseo *High* (prioridad absoluta).
 2. **SI** vida *High* → Deseo *Low* (no pierde tiempo si está sano).
 3. **SI** botín *Far* → Deseo *Low* (no cruza toda la arena exponiéndose).

Fragmento Técnico: Construcción Fluida (*Fluent Builder*):

```

1 _sharedIntercept = new FuzzyController.Builder()
2   .Samples(32)
3   .Input(distance).Input(carrying)
4   .Output(intercept)
5   // Jammo in reach and carrying a piece: go stop him.
6   .Rule().If(JAMMO_DISTANCE, "Near").And(JAMMO_CARRYING, "Yes").Then(INTERCEPT, "High")
7   // Carrying but far: not worth leaving the Player.
8   .Rule().If(JAMMO_DISTANCE, "Far").Then(INTERCEPT, "Low")
9   // Empty-handed: ignore him, stay on the Player.
10  .Rule().If(JAMMO_CARRYING, "No").Then(INTERCEPT, "Low")
11  .Build();

```

Listing 7: Definición lingüística del controlador de intercepción en BossFuzzyBrain

4.7.4 BossShield.cs

Actúa como un filtro interceptor de daño (**IDamageFilter**) anclado al componente **Health** del jefe final. Su diseño es dual: funciona como un escudo direccional clásico durante el combate, y como un sensor lógico que cuenta los impactos del jugador para liberar piezas durante la fase de recolección.

Parámetros de Inspector:

- **director** (**MirrorDuelDirector**): Referencia al director de la arena para notificarle la progresión de la mecánica.
- **hitsPerPiece** (**int**): Número de golpes exitosos (no bloqueados) necesarios para desprender una pieza. Suele coincidir con la longitud del combo del jugador (ej. 3).
- **blockAngle** (**float**): Amplitud en grados del cono frontal de bloqueo (ej. 120°).

Propiedades:

- **IsDefending** (**bool**): Indica si la guardia activa (controlada por la inteligencia artificial **BossDuelAI**) está actualmente levantada.

Métodos principales y Lógica (**IDamageFilter**):

- **SetDefending**(**bool**) y **SetShedding**(**bool**): Alternan los dos modos de funcionamiento del escudo. **SetDefending** es llamado frecuentemente por la IA (defensa reactiva o estados de guardia), mientras que **SetShedding** es invocado por el director al inicio de la fase de recolección, reiniciando el contador de golpes interno para no arrastrar impactos parciales de fases anteriores.
- **SetInvulnerable**(**bool**): Activa un bloqueo absoluto (**_blockAll**). Se utiliza exclusivamente como medida de seguridad por el director durante las transiciones cinemáticas (ej. la transición del cielo de Sol a Luna).
- **ShouldBlock**(**DamageInfo info**): Intercepta el daño milisegundos antes de que **Health** lo reciba y procese. La lógica de filtrado opera en este orden:
 1. Si **_blockAll** es verdadero, veta el daño incondicionalmente devolviendo **true**.
 2. Si el daño proviene del jugador, la guardia está levantada (**_defending**) y el ángulo del impacto entra en el cono frontal (**IsFrontal**), el golpe es bloqueado. Esto obliga al jugador a buscar la espalda del jefe o atacar cuando baje la guardia.

3. Si el golpe **supera la guardia** y la mecánica de recolección está activa (`_shedding`), suma +1 al contador. Al alcanzar `hitsPerPiece`, notifica al director mediante `NotifyPlayerBrokePiece()` y resetea el conteo a 0.
 4. Finalmente devuelve **false**, permitiendo que el componente `Health` reciba el daño (aunque durante esta fase, el director habrá limitado el daño recibido a exactamente 1 HP por golpe mediante los topes de `Health`).
- **IsFrontal(DamageInfo info)**: Método auxiliar que evalúa la direccionalidad. Calcula el vector en el plano XZ (ignorando la altura) entre el jefe y el origen del ataque, verificando si el ángulo respecto al *forward* del jefe es menor o igual a la mitad del `blockAngle`.

4.7.5 DuelHealthSpawner.cs

Spawner de **botines de vida neutrales** del duelo: en cada golpe (jugador↔jefe) puede aparecer un pickup que cura a quien lo recoja. Se engancha a `Health.Damaged` (no a `onDamaged`, que también dispara con *heal*). Mantiene un registro para la decisión difusa del jefe.

Parámetros: `playerHealth`, `bossHealth`, `pickupPrefab`, `healAmount`, `pickupLifetime`, `poolSize`, `arenaCenter`, `arenaRadius`, `spawnHeight`, `dropChance`, `minSpawnInterval`.

Métodos: `OnCombatHit(...)`, `Spawn()`, `Remove(...)`, `TryGetNearestReady(...)`.

4.7.6 MirrorFlipDirector.cs

Director especializado en la transición “celeste” que divide las dos fases del duelo final. En lugar de utilizar múltiples cámaras o cargar escenas distintas, este script orquesta una metamorfosis del entorno en tiempo real (Sol ↔ Luna). Destaca por su manipulación dinámica de materiales procedimentales y la interpolación cruzada (*crossfade*) de la iluminación, logrando un efecto de oscurecimiento y revelación continuo.

Parámetros de Inspector (Selección):

- **Cuerpos Celestes:** `celestialPivot`, `sunObject` y `moonObject`. La transición rota el pivote común usando `flipRotation` (ej. 180° en Z) para crear una órbita física, en lugar de rotar los objetos sobre su propio eje.
- **Atmósfera:** `sunSkybox`, `moonSkybox` y `reflectionSkybox`. Define los colores ambientales (`sunAmbient` / `moonAmbient`) que se interpolarán durante el giro.
- **Gestión de Capas (*Layers*):** `swapWorldLayers`, `upWorldLayer` y `downWorldLayer`. Permite intercambiar dinámicamente las capas de los astros para interactuar

con la cámara del espejo de agua (que solo renderiza el mundo superior e inferior según la capa).

- **Iluminación y Tiempos:** Luces direccionales y la duración de la transición (`blendDuration`).

Métodos principales y Lógica Visual:

- **SetupSkyBlend():** Ejecutado en el `Awake()`. Para evitar modificar permanentemente los *assets* del proyecto (lo cual ocurriría si se altera un **Material** compartido), esta función **clona** el material base del *skybox*. Cachea los identificadores de las propiedades del *shader* (`Shader.PropertyToID`) para maximizar el rendimiento.
- **FlipTo(bool moon):** Corrutina principal de la transición. Bloquea al jugador temporalmente e itera sobre un peso normalizado (`moonWeight`) aplicando una curva de suavizado (*smoothstep*). Llama secuencialmente a los actualizadores de luz, rotación y cielo. Al finalizar, invoca `DynamicGI.UpdateEnvironment()` una única vez para refrescar la iluminación global.
- **ApplySky(float moonWeight):** Interpola matemáticamente propiedades procedimentales del *skybox* clonado, como el tinte del cielo (`_SkyTint`), el grosor de la atmósfera (`_AtmosphereThickness`) y la exposición (`_Exposure`). Simultáneamente, aplica la interpolación de forma **inversa** al cielo reflejado bajo el agua, logrando que ambos cielos se crucen en el punto medio de la transición.
- **Sombreado Global (*Global Shader Properties*):** Alimenta la variable global `_BodyBlend` para que los *shaders* de los astros realicen un *crossfade* de sus propias texturas sin necesidad de invocar métodos por componente.

Fragmento Técnico: Interpolación de Propiedades Procedimentales:

```
1 // Se actualizan las propiedades del material clonado en lugar de usar un swap brusco.
2 _skyBlend.SetColor(SkyTintId, Color.Lerp(_sunTint, _moonTint, moonWeight));
3 _skyBlend.SetColor(GroundColorId, Color.Lerp(_sunGround, _moonGround, moonWeight));
4 _skyBlend.SetFloat(SunSizeId, Mathf.Lerp(_sunSunSize, _moonSunSize, moonWeight));
5 _skyBlend.SetFloat(AtmosphereThicknessId, Mathf.Lerp(_sunAtmo, _moonAtmo, moonWeight));
6 _skyBlend.SetFloat(ExposureId, Mathf.Lerp(_sunExposure, _moonExposure, moonWeight));
```

Listing 8: Interpolación en tiempo real del Skybox en MirrorFlipDirector

4.7.7 EnemySkinMirror.cs

El jefe es el doble del jugador del **sexo opuesto**: al cambiar la piel del jugador (`PlayerSkinSwitcher.SkinChanged`), el jefe cambia a la otra forma y **rebindea** su

Animator. **Parámetros:** `playerSkin`, `maleGeometry`, `femaleGeometry`. **Método:** `ApplyOpposite(bool)`.

4.8 Carpeta: `JammoCompanion`

4.8.1 `JammoCompanion.cs`

Componente transitorio de un solo uso (*one-shot*) responsable de la cinemática de activación (el “despertar”) de Jammo en el Acto 1. Orquesta la transición entre la fase de exploración libre del nivel y el inicio formal del puzzle, enlazando el sistema de diálogos con el sistema de animación antes de ceder el control definitivamente al `JammoGuideController`.

Parámetros de Inspector:

- **Conexión del Puzzle:** `puzzleManager`, referencia vital para notificar el inicio de la secuencia (`StartPuzzleSequence`).
- **Activación Cinematográfica:** `activationDialogue` (el activo de diálogo que se reproduce al interactuar), `animatorActivatedBool` (el nombre del parámetro en el *Animator*, por defecto "IsActivated") y `getUpDelay` (tiempo sincronizado con la duración de la animación de levantarse).
- **Bloqueo del Jugador:** `cinematicManager`, utilizado para congelar el movimiento del jugador mientras observa a Jammo despertarse (permitiendo mantener el control de la cámara para no desorientar al usuario).

Métodos principales y Lógica:

- `Start()`: Fase de preparación. Recupera y desactiva inmediatamente el `NavMeshAgent` de Jammo para garantizar que no reaccione a colisiones físicas ni intente perseguir al jugador libremente antes del puzzle.
- `HandleActivationInteraction()`: Punto de entrada enlazado al *trigger* de la zona de Jammo. Está protegido por una bandera (`_hasBeenTriggered`) para garantizar una única ejecución. Si hay un diálogo asignado, invoca al `DialogueManager` pasándole el inicio de la corrutina como una **función anónima** (*callback*) a ejecutar cuando el diálogo concluya.
- `GetUpAndActivateRoutine()`: Corrutina que ejecuta la secuencia de forma estricta:
 1. Bloquea el movimiento del jugador.

2. Dispara la animación de despertar en el *blend tree* de Jammo.
3. Espera en tiempo real los segundos dictados por `getUpDelay`.
4. Llama a `puzzleManager.StartPuzzleSequence()`, transfiriendo la autoridad del nivel. Incluye un mecanismo de seguridad (*fallback*) que desbloquea al jugador si, por error de configuración, el *manager* no estuviera asignado.
5. **Autodestrucción lógica:** Finalmente, el script se desactiva a sí mismo (`this.enabled = false`), dejando paso libre al `JammoGuideController` para manejar el movimiento del robot.

Fragmento Técnico: Callbacks en Diálogos y Autodesactivación:

```

1  // Inicia el dialogo y pasa la corrutina como callback al finalizar
2  if (activationDialogue != null && DialogueManager.Instance != null)
3  {
4      DialogueManager.Instance.PlayDialogue(
5          activationDialogue,
6          () => StartCoroutine(GetUpAndActivateRoutine())
7      );
8  }
9
10 // ... (Dentro de GetUpAndActivateRoutine) ...
11
12 // START THE PUZZLE!
13 if (puzzleManager != null) puzzleManager.StartPuzzleSequence();
14
15 // Disable this script because Jammo is now controlled by JammoGuideController!
16 this.enabled = false;

```

Listing 9: Manejo de la interacción y delegación de control en `JammoCompanion`

4.8.2 JammoActivationTrigger.cs

Volumen *trigger* de un solo uso: invoca un `UnityEvent` (`onPlayerEnter`) la primera vez que el jugador entra (p. ej. despertar a Jammo). Requiere `Collider`.

4.9 Carpeta: Items

4.9.1 InteractableObject.cs

Interactuable genérico: un *trigger* que muestra un aviso mientras el jugador está cerca e invoca un `UnityEvent` (`onInteract`) al pulsar F. Los comportamientos se cablean por objeto en el Inspector (sin acoplamiento). **Parámetros:** `promptIcon`, `onInteract`.

4.9.2 InteractionUIManager.cs

Singleton de la “nuvoletta” de interacción en *Screen Space*: sigue al jugador con `Camera.WorldToScreenPoint` y aparece/desaparece con *fade*. **Parámetros:** `promptCanvasGroup`, `promptRectTransform`, `promptImage`, `mainCamera`, `screenOffset`, `fadeSpeed`. **Métodos:** `ShowPrompt(Transform, Sprite)`, `HidePrompt()`, `FadeRoutine(...)`.

4.9.3 Items/Objects/BookManager.cs

Gestiona la UI del libro del prólogo: la primera interacción “recoge” el libro; después se abre/cierra con B/F. Abrir bloquea al jugador; el primer cierre puede disparar un diálogo. **Parámetros:** `bookPanel`, `player`, animación (`animationDuration`, `offScreenPosition`, `onScreenPosition`), `worldBookObject`, `startAlreadyPickedUp`, `postBookDialogue`, `onBookPickedUp`; `IsOpen` (propiedad).

Métodos: `ToggleBookMenu()`, `CloseAndMaybeShowDialogueRoutine()`, `SlideRoutine(...)`, `SetPlayerMovement(bool)`.

4.9.4 Items/Objects/CinematicFallManager.cs

Cinématica de caída del jugador a la tumba siguiendo la receta del proyecto: bloqueo de input → cámara cenital de Cinemachine → corrutina que conduce un **shader de disolución global** → restaurar. **Parámetros:** `topDownCamera`, `player`, `environmentToFall`, `targetYPosition`, `fallDuration`, `objectsToVanish`, `particlesToStop`, `onFallStarted`; `IsPlaying` (propiedad).

Métodos: `StartFallSequence()`, `FallRoutine()` y `SetPlayerMovement(canMove, keepLookActive)` —el bloqueo de jugador reutilizado por otros sistemas.

4.9.5 Items/Objects/HealthPickup.cs

Botín de vida soltado por el `WaveManager` al morir un enemigo (cura al jugador o a Jammo según `healTarget`). Apto para pooling (callback de propietario, sin `Destroy`). **Parámetros:** `healTarget`, `healAmount`, `playerTag`, `onCollected`; evento estático `Collected(bool healedPlayer)` (puntuación).

Métodos: `Configure(Action)`, `OnTriggerEnter(...)`, `ResolveHealTarget(...)`.

4.9.6 Items/Objects/DuelHealthPickup.cs

Botín de vida **neutral** del duelo final: cura a quien lo pise (jugador o jefe), con las dos `Health` válidas inyectadas por el spawner. Apto para pooling. **API:** `Configure(onRemoved, a, b, healAmount, lifetime)`, propiedad `SpawnTime` (para el retardo de reacción del jefe); `OnTriggerEnter`, `LifetimeRoutine`, `Remove`.

4.10 Carpeta: Dialogue

4.10.1 DialogueLine.cs

struct serializable de una línea: `speaker` (string) y `text` (string, [TextArea]).

4.10.2 DialogueAsset.cs

ScriptableObject (menú “Vellum/Dialogue Asset”) con `dialogueId` y `lines` (List<DialogueLine>).

4.10.3 DialogueManager.cs

Singleton que reproduce DialogueAsset: *fade* del lienzo, máquina de escribir (Espacio salta) y **bloqueo del jugador** durante el diálogo. **Parámetros:** `dialogueCanvasGroup`, `speakerLabel`, `bodyText`, `advanceHint`, `player`, `fadeSpeed`, `charsPerSecond`; `IsPlaying` (propiedad).

Métodos: `PlayDialogue(DialogueAsset, Action)`, `LockPlayer()/UnlockPlayer()` (para directores externos), `RunDialogue(...)`, `TypewriterRoutine(...)`, `WaitForSpace()`, `SetPlayerLocked(bool)`.

4.11 Carpeta: UI

4.11.1 PlayerHUD.cs

Barra de vida del jugador. Se engancha a `Health.onDamaged` (normalizado) desde el Inspector y anima el relleno con degradado y suavizado. **Parámetros:** `playerHealth`, `fillImage`, `fillGradient`, `valueLabel`, `smoothDuration`. **Métodos:** `UpdateBar(float)`, `SmoothTo(...)`, `Apply(...)`.

4.11.2 HudReveal.cs

Slide desde abajo + *fade* para un elemento de UI; `Reveal()/Hide()` públicos e idempotentes (usados por los directores y la pausa). **Parámetros:** `canvasGroup`, `slideDistance`, `duration`, `hiddenOnStart`; `IsVisible` (propiedad). **Métodos:** `Reveal()`, `Hide()`, `Animate(...)`.

4.11.3 StatueProgressBar.cs

Barra vertical de progreso de la estatua (Acto 2). Se engancha a `StatueRig.onPartRevealed` y lee `StatueRig.Normalized`; incluye un “menisco” para la curvatura del líquido.

Parámetros: `statueRig`, `fillImage`, `fillGradient`, `meniscusRect`, `valueLabel`, `showPercentage`, `smoothDuration`. **Métodos:** `UpdateBar()`, `SmoothTo(...)`, `Apply(...)`.

4.11.4 `MirrorCameraSync.cs`

Refleja una cámara sobre el plano de agua (Acto 3): refleja posición y rotación respecto a `waterLevel` y sincroniza el FOV. **Parámetros:** `mainCamera`, `waterLevel`. **Método:** `LateUpdate()`.

4.11.5 `FloorProximityManager.cs`

Envía la posición del jugador al material del suelo cada frame (efecto de brillo por proximidad), sobre un material instanciado. **Parámetros:** `player`. **Método:** `Update()` (`SetVector "_PlayerPos"`).

4.11.6 `CRTController.cs`

Gestiona el filtro CRT retro como `Image`, sincronizando con `PlayerPrefs` ("CRT-Filter"). **Métodos:** `Awake()` (cachea la `Image`), `Start()` (lee la preferencia), `UpdateFilter(bool)` (habilita/deshabilita el render).

4.11.7 `SettingsUI.cs`

Controlador principal de la interfaz de opciones del menú. Gestiona las preferencias de audio mediante el sistema de `AudioMixer` de Unity y las configuraciones de video (pantalla completa y filtro retro CRT), garantizando la persistencia de los datos entre sesiones mediante `PlayerPrefs`.

Parámetros de Inspector:

- **Motor de Audio:** `audioMixer`, referencia al *mixer* principal del juego.
- **Controles de UI:** `musicSlider`, `sfxSlider` y `muteToggle` para el sonido; `fullscreenToggle` y `crtToggle` para el apartado visual.

Métodos principales y Lógica:

- `SetupAudioUI()` y `SetupVideoUI()`: Recuperan los valores almacenados en `PlayerPrefs` al iniciar la escena. Utilizan los métodos `SetValueWithoutNotify` y `SetIsOnWithoutNotify` de Unity UI. Esto es una buena práctica crucial: permite actualizar el estado visual del control sin disparar el evento `onValueChanged`, evitando así ruidos de *feedback* indeseados o llamadas recursivas durante la inicialización.

- `SetMusicVolume(float)` y `SetSFXVolume(float)`: Capturan el valor lineal del *slider* (típicamente de 0.0001 a 1) y lo convierten a la escala logarítmica (decibelios, de -80dB a 0dB) requerida por el `AudioMixer` usando la fórmula matemática $\text{dB} = \log_{10}(\text{valor}) \times 20$.
- `SetMute(bool)`: Actúa como un interruptor maestro modificando directamente `AudioListener.volume` entre 0 y 1.
- `SetFullscreen(bool)`: Alterna el modo de visualización comunicándose con `Screen.fullScreen`.
- `SetCRT(bool)`: Método global que interroga la escena buscando todos los `CRTController` instanciados (incluso aquellos temporalmente desactivados, utilizando `FindObjectsInactive.Include`) para aplicar el filtro de post-procesado retro a todas las cámaras simultáneamente.

Fragmento Técnico: Conversión de Audio e Inicialización Silenciosa:

```

1 void SetupAudioUI()
2 {
3     float savedMusic = PlayerPrefs.GetFloat("MusicVolume", 1f);
4     // SetValueWithoutNotify actualiza la UI sin disparar el evento (evita ruidos al cargar)
5     if (musicSlider != null)
6     {
7         musicSlider.SetValueWithoutNotify(savedMusic);
8         musicSlider.onValueChanged.AddListener(SetMusicVolume);
9     }
10    SetMusicVolume(savedMusic);
11 }
12
13 public void SetMusicVolume(float value)
14 {
15     // Conversion de escala lineal a logaritmica (Decibelios)
16     audioMixer.SetFloat("MusicVol", Mathf.Log10(value) * 20f);
17     PlayerPrefs.SetFloat("MusicVolume", value);
18 }

```

Listing 10: Buenas prácticas en la gestión de UI y Audio en `SettingsUI`

4.11.8 GlobalButtonStyling.cs

Sistema automático que unifica la estética de **todos los botones**: extrae colores del *sprite* del botón y genera capas de *hover/pressed* animadas, sin configurar animaciones a mano. **Parámetros:** `hoverPixelUV/pressedPixelUV`, `animationSpeed`, `fillPadding`, `buttonHoverScale/ buttonPressedScale`, `hoverAlpha/pressedAlpha`. **Métodos:** `Start()` (escanea botones), `ApplyAnimatedHoverLogic(Button)` (reingeniería con capas `HoverFill/PixelArt_Graphic`), `GetColorFromSprite(...)` (requiere *Read/Write*), `ApplyScaleOnlyHover(...)`. Las clases `HoverAnimator` y

`SimpleHoverScale` implementan las interfaces de `EventSystems` para escalar las capas con `Time.unscaledDeltaTime`.

4.11.9 `ButtonStyleOverride.cs`

Marcador: los botones con este componente son saltados por `GlobalButtonStyling` (o solo conservan el escalado). **Parámetros:** `keepHoverScale`, `hoverScale`, `animationSpeed`.

4.12 Carpeta: Score

4.12.1 `RunStats.cs`

Modelos de datos (`[Serializable]`) de las estadísticas de una partida: `GlobalStats`, `LevelOneStats`, `LevelTwoStats`, `LevelThreeStats` y el contenedor `RunStats` (incluye los scores calculados `level1Score...finalScore` y `grade`). Diseñados para `JsonUtility`.

4.12.2 `Leaderboard.cs`

Sistema de persistencia local para el registro de puntuaciones máximas. Utiliza el formato JSON para empaquetar objetos complejos (`LeaderboardEntry`), incluyendo el desglose completo de estadísticas (`RunStats`), y los almacena en el registro persistente del sistema mediante `PlayerPrefs`.

Estructuras de datos:

- **`LeaderboardEntry`** (`Serializable`): Almacena la información de cada partida: nombre del jugador, calificación (*grade*), puntuación numérica, el objeto `RunStats` completo (que contiene el desglose por niveles y tiempos) y una marca de tiempo en formato ISO 8601.
- **`LeaderboardData`** (`Serializable`): Clase envoltorio (**wrapper**). Es necesaria porque `UnityEngine.JsonUtility` no puede serializar listas de nivel superior de forma directa; el envoltorio permite la serialización jerárquica.

Métodos principales de `LeaderboardStore` (Utilidad Estática):

- **`Load()`**: Recupera el JSON almacenado bajo la clave `"vellum.leaderboard"`. Incluye una estructura de *try-catch* para manejar casos de corrupción de datos; si el parseo falla o los datos son incompatibles (ej. versiones antiguas del esquema), el sistema reinicia la tabla en lugar de bloquear el juego.

- **Add(LeaderboardEntry, int):** El punto de entrada principal para guardar nuevas partidas. Inserta la nueva entrada, ordena la lista de forma descendente usando un predicado *lambda* (`b.finalScore.CompareTo(a.finalScore)`) y recorta la lista al tamaño máximo permitido ($N = 10$ por defecto) antes de confirmar la persistencia en disco con `PlayerPrefs.Save()`.
- **Save(LeaderboardData):** Convierte el objeto a cadena JSON y actualiza el registro local.

Consideración Técnica: Serialización en Unity: Al utilizar `JsonUtility`, el sistema depende estrictamente de que todas las clases serializadas tengan el atributo `[Serializable]`. Esto asegura un rendimiento mucho mayor que el uso de librerías de reflexión pesadas (como *Newtonsoft.Json*), siendo ideal para plataformas móviles o consolas donde el acceso a disco debe minimizarse. La persistencia mediante `PlayerPrefs` garantiza que las tablas de puntuación sobrevivan a cierres inesperados de la aplicación.

4.12.3 ScoreManager.cs

Actúa como el motor de persistencia y cálculo estadístico para toda la sesión de juego. A diferencia de otros directores, es un **Singleton auto-creado** mediante el atributo `[RuntimeInitializeOnLoadMethod]`. Esto elimina la necesidad de colocar manualmente el objeto en cada escena, garantizando que el `ScoreManager` exista desde el primer *frame* y sobreviva a los cambios de nivel gracias a `DontDestroyOnLoad`.

Modelo de Datos y Funciones:

- **Gestión de Tiempos:** Utiliza temporizadores basados en `Time.deltaTime`, lo que implica que el tiempo de pausa (donde `timeScale=0`) no se suma a la estadística, penalizando únicamente el tiempo activo de juego.
- **Modelo Aditivo de Puntuación:** Implementa el modelo definido en *docs/punteggio.md*. El método `ComputeScores()` consolida los datos crudos en tres puntuaciones nivel-específicas (`level1Score, ...`) y un bono global calculado a partir de la precisión (*accuracy*) y la salud perfecta.
- **Persistencia (*Snapshotting*):** Al finalizar una partida, `SaveEntry(string)` realiza una **copia profunda** (*deep copy*) de las estadísticas actuales mediante serialización JSON. Esto es crítico: garantiza que el registro histórico en la tabla de puntuaciones sea inmutable y no se vea alterado si el jugador comienza una nueva partida (`ResetRun()`).

API de Estadísticas: El sistema provee una API granular para que cualquier componente del juego (enemigos, directores, triggers) notifique eventos sin necesidad de acoplamiento rígido:

- **Combate:** `AddSwing`, `AddHit`, `AddDamageDealt`, `BreakStreak`.
- **Progresión por Acto:** `AddFallen` (Nivel 1), `AddStatuePartPlaced` (Nivel 2), `AddPieceDelivered` (Nivel 3).
- **Cálculo de Notas:** `GradeFor(int)` mapea el puntaje final a una calificación sintética (S, A, B, C, D) mediante umbrales constantes, facilitando la retroalimentación inmediata al jugador en la pantalla de resultados.

Herramientas de Depuración: Incluye métodos marcados con `[ContextMenu]` (accesibles haciendo clic derecho en el componente desde el Inspector de Unity). Esto permite al equipo de desarrollo:

- **Simular partidas:** `DebugFillSampleRun` permite probar el cálculo de notas sin jugar.
- **Sembrar datos:** `DebugSeedSampleEntries` inyecta tres perfiles variados (“ALEX” como excelente, “JAMIE” como intermedio y “SAM” como fracasado), facilitando el diseño y previsualización de la interfaz de la tabla de puntuaciones sin necesidad de terminar el juego.

4.12.4 `ScoreLevelsBinder.cs`

Binders que **alimentan** al `ScoreManager` desde los eventos de cada Acto. Se colocan en la escena y se suscriben en `OnEnable/OnDisable` (sin *polling*). El de nivel 1 engancha los `UnityEvent` del `PathPuzzleManager`; el de nivel 2 engancha `WaveManager/StatueRig/Health/PlayerMeleeAttack/ FrontalShieldBlock/ HealthPickup.Collectected`; el de nivel 3 engancha `MirrorDuelDirector` y las `Health` del jugador/`Jammo/jefe` (para el crédito parcial en derrota).

4.12.5 `LeaderboardUI.cs`

Construye la lista de la tabla: limpia el contenedor, carga las entradas persistidas y instancia un `LeaderboardEntryView` por entrada. **Parámetros:** `entryPrefab`, `contentParent`, `emptyLabel`. **Método:** `Refresh()` (soporta prefab de proyecto o plantilla en escena).

4.12.6 LeaderboardEntryView.cs

Vista de una fila: fila principal (rango/nombre/score/nota) + panel de detalles en acordeón con cuatro bloques (global/L1/L2/L3). **Parámetros:** textos y detailsButton, detailsPanel, los cuatro TMP_Text de estadísticas.

Métodos: Populate(rank, entry), ToggleDetails(), ShowSection(int) y los formateadores

BuildGlobal/BuildL1/BuildL2/BuildL3.

4.13 Carpeta: Audio

4.13.1 SoundManager.cs

Reproducción central de audio: SFX *one-shot* y música en bucle, enrutados por grupos del AudioManager (controlados por los *sliders* de Ajustes). Singleton DontDestroyOnLoad.

Parámetros: sfxGroup, musicGroup. **Métodos:** PlaySfx(clip, volume), PlayMusic(clip), StopMusic().

4.13.2 AudioCue.cs

Disparador de SFX reutilizable: contiene clips y reproduce uno por SoundManager. Se cablea a cualquier UnityEvent o Animation Event. **Parámetros:** clips, volume, randomize. **Métodos:** Play(), PlayIndex(int).

4.13.3 LevelMusic.cs

Reproduce la música de fondo de la escena en Start (vía SoundManager). **Parámetros:** music.

4.13.4 PlayerCombatAudio.cs

Reproduce los SFX de combate del jugador suscribiéndose a los eventos C# melee.HitLanded y shield.Blocked (no cableables por Inspector). **Parámetros:** melee, shield, hitCue, blockCue.

4.14 Carpeta: Menu

4.14.1 MainMenuManager.cs

Controlador maestro de la interfaz de usuario que orquesta el menú principal, la pausa durante el juego y la pantalla de fin de partida. Gestiona la transición entre los estados de juego y menú mediante transiciones de cámara cinematográficas (usando

Cinemachine) y efectos visuales de desenfoque (*Post-Processing*), bloqueando el *input* del jugador y manipulando el `Time.timeScale` para pausar la simulación.

Parámetros de Inspector (Selección):

- **Gestión Cinematográfica:** Referencias a `menuCamera` y `cameraBrain`. Permite la transición fluida entre la vista del menú y la perspectiva del jugador.
- **UI y Subpaneles:** `menuUIContainer` (contenedor raíz), `settingsPanel` y `leaderboardPanel`. `hudBars` es un arreglo de componentes `HudReveal` que permite ocultar/mostrar elementos de HUD (como la salud) de forma selectiva durante la pausa.
- **Pantalla de Fin de Juego:** `deathContainer` es un contenedor compartido para muerte y victoria, diferenciado mediante los objetos `gameOverImage` y `youWinImage`. Incluye campos de entrada de texto (`TMP_InputField`) para registrar el nombre del jugador en el `Leaderboard`.
- **Bloqueos de Seguridad:** Referencias a `bookManager`, `cinematicFallManager` y `jammoGuideController`. Estos actúan como *gates*: si alguno de estos sistemas está activo, el menú de pausa mediante la tecla `Esc` se bloquea para evitar conflictos con cinemáticas o menús contextuales (ej. lectura de libros).

Métodos principales y Lógica:

- **Transiciones de Juego/Menú:** Los métodos `TransitionToGame()` y `TransitionToMenu()` ejecutan una corrutina que interpola la posición, el *alfa* de la UI y el peso del volumen de post-procesado (`menuBlurVolume`). Esto crea una transición visualmente coherente donde el fondo se desenfoca mientras el menú aparece.
- **Manejo del cursor y tiempo:** Al pausar, fuerza el estado del cursor a `CursorLockMode.None` y establece `Time.timeScale = 0f`. Al reanudar, devuelve el control al jugador, bloquea el cursor y utiliza `LockCursorDeferred()` para evitar que el *input* del ratón reaccione al último clic de botón en el menú.
- **Fin de Juego (*GameOver/Victory*):** Los métodos `ShowGameOver()` y `ShowVictory()` unifican la lógica. Ambos llaman a `ShowEndScreen()`, que detiene el juego y abre el contenedor final. Si el jugador pulsa “Guardar Puntuación”, se utiliza el `ScoreManager.Instance.SaveEntry()` para persistir los datos de la partida.
- **Gestión del HUD:** Antes de abrir el menú, `HideHudBars()` recorre todos los elementos definidos y guarda su estado de visibilidad en un array de booleanos (`_hudWasVisible`). Al reanudar, `RevealHudBars()` restaura solo aquellos

elementos que estaban activos originalmente, evitando errores lógicos si, por ejemplo, la barra de salud estaba oculta antes de pausar.

Nota técnica sobre la persistencia y carga: El método `RestartGame()` es un punto crítico: antes de recargar `Act_01`, fuerza el reset de la escala de tiempo, limpia el radio del *shader* global (para evitar que el efecto de disolución final del Acto 3 persista al reiniciar) e invoca `ScoreManager.Instance.ResetRun()`, asegurando que la nueva partida comience con estadísticas limpias.

4.15 Carpeta: Utils

4.15.1 SimplePool.cs

Pool de instancias de prefab reutilizables (sin `Destroy`): los objetos liberados se desactivan y se reusan. **API:** `Get(position, rotation)`, `Release(GameObject)`.

4.15.2 AnimatorParameterCache.cs

Cachea los parámetros que existen en un `Animator` para evitar advertencias “Parameter does not exist” con controladores que no los tengan. **API:** `Refresh(Animator)`, `Has(int hash)`.

4.15.3 TileGridGenerator.cs

Utilidad de Editor que genera proceduralmente la rejilla de baldosas del Acto 1 sobre un *Plane*, más un anillo decorativo redondeado. **Parámetros:** `tilePrefab`, `floorPlane`, `tileSize`, `decorativeTilePrefab`, `decorativeRings`. **Métodos ([ContextMenu]):** `GenerateGrid()`, `GenerateDecorativeRing()`.

4.15.4 UIRaycastDebugger.cs

Ayuda de depuración: al hacer clic, lanza un *raycast* de UI bajo el cursor y registra cada `Graphic` golpeado, avisando de elementos totalmente transparentes que puedan bloquear clics.

4.15.5 Utils/Editor/ForestSower.cs

Utilidad de Editor que esparce prefabs de árboles en un anillo, ajustándolos al terreno con un *raycast* y aleatorizando rotación/escala. **Parámetros:** `treePrefabs`, `minRadius/maxRadius`, `count`, `minScale/maxScale`, `terrainLayer`. **Métodos ([ContextMenu]):** `GenerateForest()`, `Clear()`.

4.15.6 Utils/Editor/PathTileSetupTool.cs

Herramienta de menú de Editor (Tools/Vellum/...) que enlaza el campo `solidCollider` de cada `PathTile` a su único `BoxCollider` no-trigger. Idempotente, con grupo de *Undo*.

4.15.7 Utils/Editor/SceneViewCapture.cs

Utilidad de captura de pantalla del Editor (atajos `Ctrl/Cmd+Shift+K/J`): captura el *Scene View* (WYSIWYG) o la *Main Camera* a 1080p, usando *render requests* de la SRP (URP). Guarda en <Proyecto>/Screenshots/.

4.15.8 PathPuzzleManagerEditor.cs

Inspector personalizado de `PathPuzzleManager` que añade un botón “**Generate Path!**” para previsualizar un camino en el Editor (llama a `GenerateRandomPath()` con *Undo*).

5 Agradecimientos y Referencias de Activos

Para el desarrollo de este proyecto, además del código base implementado de forma nativa, se han utilizado diversos recursos externos. Muchos elementos de la interfaz de usuario (UI), como los botones y paneles, han sido diseñados específicamente para *Vellum* mediante scripts personalizados y generadores procedurales ubicados en el directorio `Utils/CSS`. El resto de los activos se detallan a continuación.

Nombre del Activo / Recurso	Enlace / Referencia
<i>Activos Unity / Gráficos</i>	
3D set of stylized nature - GHIBLI style	Unity Asset Store
AllSkyFree (Skybox Set)	Unity Asset Store
Simple Gems and Items Ultimate	Unity Asset Store
Modular Dungeon Pack	Unity Asset Store
Jammo-Character	Unity Asset Store
Lowpoly Baker's House	Unity Asset Store
Meadow Forest Props	Unity Asset Store
Low Poly Fantasy Village	Unity Asset Store
Lowpoly Environment Pack	Unity Asset Store
RPG Monster DUO PBR Polyart	Unity Asset Store
RPG Tiny Hero Duo	Unity Asset Store
RPG Monster Partners PBR Polyart	Unity Asset Store
Simple Nature Pack	Unity Asset Store
Starter Assets (URP)	Unity Asset Store
<i>Activos de Audio</i>	
Scott Buckley - Ephemera	YouTube
Scott Buckley - The Climb	YouTube
Best Sword Sounds (Slice/Slash/Crash/Swoosh)	YouTube
Shield Block Free Sound Effect	YouTube
Cracks sound effects library	YouTube
Looperman - Drone FX	Looperman
Looperman - Droid talking SFX	Looperman
Looperman - PLT Layer A	Looperman
Looperman - DDS Layer C	Looperman
Looperman - CCE Layer B	Looperman
Wooden Treasure Chest Opening	YouTube
Slime Squishes and Oozes	YouTube
Twist And Break Spine Sound Effect	YouTube
Door Slam sound effect	YouTube
Looperman - Trance Synth Layer 02	Looperman
Looperman - Alien Bees	Looperman
Heal Sound Effect	YouTube
Looperman - Hell on Earth Riser	Looperman
SavfkMusic - For Tomorrow	YouTube
Looperman - FX 014 Impact One Shot	Looperman
Looperman - Laser	Looperman

Tabella 1: Lista detallada de activos y recursos externos utilizados.